

PentestArena — Guia de Implementação

O que é este documento: um **guia de implementação** completo do PentestArena — não é código, é a especificação operacional que você segue para *construir* o sistema, peça por peça, de forma que **conecte e funcione sem bugs**. Contém: o conceito, os contratos de dados que ligam os módulos, a arquitetura, os catálogos (técnicas, ferramentas, arcanos), as telas, e os **sprints prontos para colar** no assistente.

Parte 0 — Como usar este guia (workflow Codex + fable no VSCode)

Divisão de trabalho na implementação:

- **fable (eu), no VSCode** — faz o trabalho de engenharia: arquitetura, a lógica de todos os módulos, a **integração entre as peças** (é onde bugs aparecem), os contratos de dados, os testes e a depuração. Use-me para escrever/revisar código, ligar um módulo ao outro, e garantir que o fluxo roda ponta a ponta sem quebrar. Trabalhe **um sprint por vez** (Parte 6/roadmaps): eu implemento, os testes daquele sprint passam, e só então seguimos.
- **Codex** — faz os **assets visuais**: os 22 sprites de tarô (Parte 5). Opcionalmente, boilerplate repetitivo (ex: gerar as 22 pastas-carta a partir de um template). O Codex não decide arquitetura nem integra módulos — ele produz peças que eu conecto.

Regra de ouro do processo: nunca "implemente tudo de uma vez". Cada sprint entrega algo **testável e rodando**. Antes de avançar, os testes do sprint passam e o fluxo daquele pedaço funciona de verdade contra o Juice Shop local. É assim que o projeto fica operacional e sem bugs de integração.

Ordem lógica de leitura (a numeração cresceu com o projeto; leia nesta ordem): Parte 0 → Enquadramento ético → Parte 1 (conceito) → **Parte 0.1 (contratos — o mais importante para não ter bug)** → Parte 2 (arquitetura e loop) → Parte 3 (pastas) → Parte 5 (tarô/sprites) → Parte 6 + roadmaps de UI (sprints, na ordem) → Partes 8–13 (segurança/VM/VPN, lacunas, binário, artigo, literatura, telas) → Checklist final.

Parte 0.1 — Os contratos que conectam tudo (leia antes de codar)

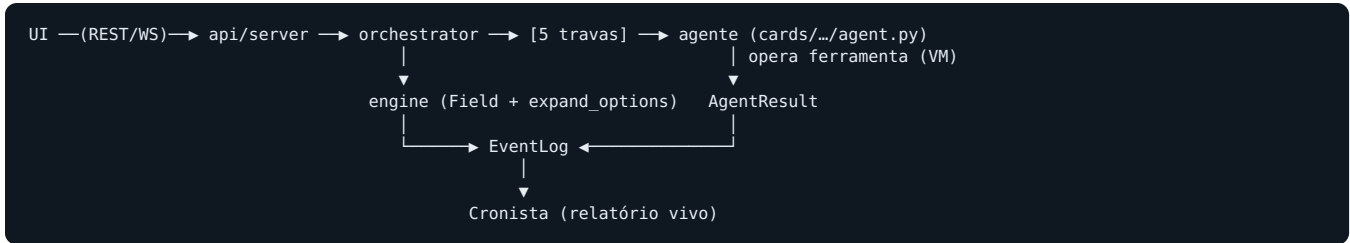
Bugs de integração nascem quando dois módulos "conversam" com formatos diferentes. Para evitar isso, **cinco contratos** são a única fronteira entre os componentes. Se todo módulo respeitar exatamente estes formatos, as peças encaixam sem surpresa. (Os detalhes completos estão nas seções citadas; aqui é o mapa.)

1. **Card** (schema 2.3) — a definição declarativa de uma carta (`id`, `arcana`, `badge`, `tool`, `preconditions`, `effects`, `attack` / `owasp`). Fonte: `cards/**/card.yaml`. Consumido por: registry, engine, UI.
2. **AgentResult** (schema 2.3 + 2.19) — o que **todo** agente devolve: `status`, `findings`, `state_delta`, `activity[]` (stream ao vivo) e `artifacts[]` (tipados). É o contrato que faz agentes heterogêneos caberem numa UI só. Produzido por: agentes. Consumido por: orquestrador, UI, Cronista.
3. **Event** (EventLog, 2.5) — tudo que acontece vira um evento append-only em JSONL (`CardPlayed`, `FindingAdded`, `VpnDropped`, `PhaseTransition`...). É a **fonte da verdade**: o `Field` e o relatório são reconstruíveis só a partir dele. Produzido por: orquestrador/engine. Consumido por: engine (replay), Cronista, Replay UI.
4. **Field** (2.7) — o estado do alvo em memória, sempre reconstruível do EventLog. `expand_options(Field)` é **puro e determinístico** (dada a seed). Produzido por: engine. Consumido por: engine (mão jogável), UI (campo).
5. **Mensagens WebSocket** (2.19) — o que o backend transmite ao vivo à UI: eventos + `ActivityEvent`s + `artifacts`. Um único canal tipado; o frontend despacha por tipo.

As 5 travas invioláveis (o que impede bug e uso indevido), aplicadas SEMPRE no orquestrador antes de qualquer agente:

1. Alvo ∈ allowlist, senão `blocked_by_allowlist`.
2. VPN conectada para ferramentas `net:true`, senão `blocked_by_vpn_down`.
3. Orçamento (mana/requisições/tempo) disponível, com timeout por agente.
4. Ferramenta instalada, senão `tool_missing` (nunca falha silenciosa).
5. Fatos do relatório vêm do EventLog, nunca do LLM.

Mapa de dependências (quem chama quem):



Se cada seta respeitar o contrato da fronteira, a integração "simplesmente funciona". É por isso que os sprints constroem **de baixo para cima** (contratos e engine primeiro, UI por último) — assim cada camada testa contra uma fronteira já estável.

[!] Enquadramento ético e legal (leia primeiro)

Este projeto executa técnicas de pentest **exclusivamente** contra alvos controlados e autorizados: - Alvos permitidos: `localhost`, containers de laboratório, apps deliberadamente vulneráveis (OWASP Juice Shop, DVWA, WebGoat), ou domínios com autorização explícita por escrito. - O sistema deve ter uma **allowlist de alvos** codificada. Qualquer alvo fora dela é recusado pelo orquestrador antes de qualquer agente rodar. - Todo o log de ações fica registrado (o "agente documentador"), o que também serve de trilha de auditoria.

Para o artigo científico, deixe isso explícito na seção de metodologia. É o que separa "pesquisa em segurança ofensiva" de "uso indevido".

PARTE 1 — Conceito

1.1 A metáfora → mapeamento técnico

O jogo é uma metáfora navegável de um fluxo de pentest orquestrado por agentes. Cada elemento do "RPG de cartas" corresponde a um componente real do pipeline:

Elemento do jogo	Significado técnico
Inimigo / Boss	O alvo de teste (link/app numa allowlist). Tem "vida" = superfície de ataque ainda não mapeada, e "defesas" = controles de segurança encontrados.
Campo de batalha	O <i>game state</i> : o que já se sabe do alvo agora (portas abertas, tecnologias, endpoints, achados). Muda a cada rodada.
Carta	Um agente especializado pré-configurado que testa um aspecto (ex: RECON-Portas, WEB-SQLi, WEB-XSS, AUTH-BruteForce, CONN-TLS). Jogar a carta = executar o agente.
Mão do jogador	O subconjunto de cartas relevantes ao estado atual . Só aparecem na mão as cartas cujos pré-requisitos foram satisfeitos (ex: WEB-SQLi só entra na mão depois que RECON-Web achou um formulário/param).
Baralho / Coleção	Todos os agentes-carta disponíveis no sistema.
Mana / Custo	Orçamento de execução (tempo, requisições, risco). Evita que o agente rode tudo indiscriminadamente.
Dano / Efeito	O resultado do agente: achados (findings), novos endpoints revelados, credenciais, etc. — que atualizam o campo.
Cronista (NPC fixo)	O agente documentador : observa cada jogada e escreve o relatório final estruturado.
Fase / Turno	Uma etapa do pentest (Reconhecimento → Enumeração → Exploração → Pós-exploração → Relatório).

1.2 O loop central — a mecânica exata (leia com atenção)

Este é o ponto mais importante do jogo, e precisa ficar cristalino: **a carta É o agente**, mas ela existe em dois momentos distintos.

- A carta como OPÇÃO no campo (antes de jogar):** o campo de batalha mostra, a cada momento, **todas as opções de exploração possíveis dado o que já se sabe do alvo**. Cada opção é uma carta = um agente disponível para agir *sobre um alvo concreto já descoberto*. Ou seja: o campo não mostra "a carta SQLi" em abstrato — ele mostra

"SQLi contra o parâmetro `q` do endpoint `/rest/products` que acabamos de descobrir". Essas opções aparecem **automaticamente** conforme o campo evolui (é o "motor de completção", seção 2.7).

2. **A carta como AÇÃO (ao jogar):** quando você **seleciona e usa** uma carta, o **agente entra em ação** — executa a técnica contra aquele alvo específico. O **resultado do agente altera o campo de batalha**: revela novos endpoints, portas, vulnerabilidades — e isso, por sua vez, **faz surgirem novas opções/cartas** no campo. O ciclo recomeça.

Em uma frase: o campo mostra automaticamente as explorações possíveis (cartas-opção); você escolhe uma; o agente daquela carta age sobre o alvo; o resultado muda o campo; o campo recalcula e revela as próximas opções — com o cronista registrando cada passo, até o objetivo ou o fim do orçamento.

É um loop de **descoberta** → **opções** → **ação** → **mutação** → **novas opções**. As cartas na sua mão são sempre o reflexo direto do estado atual do campo.

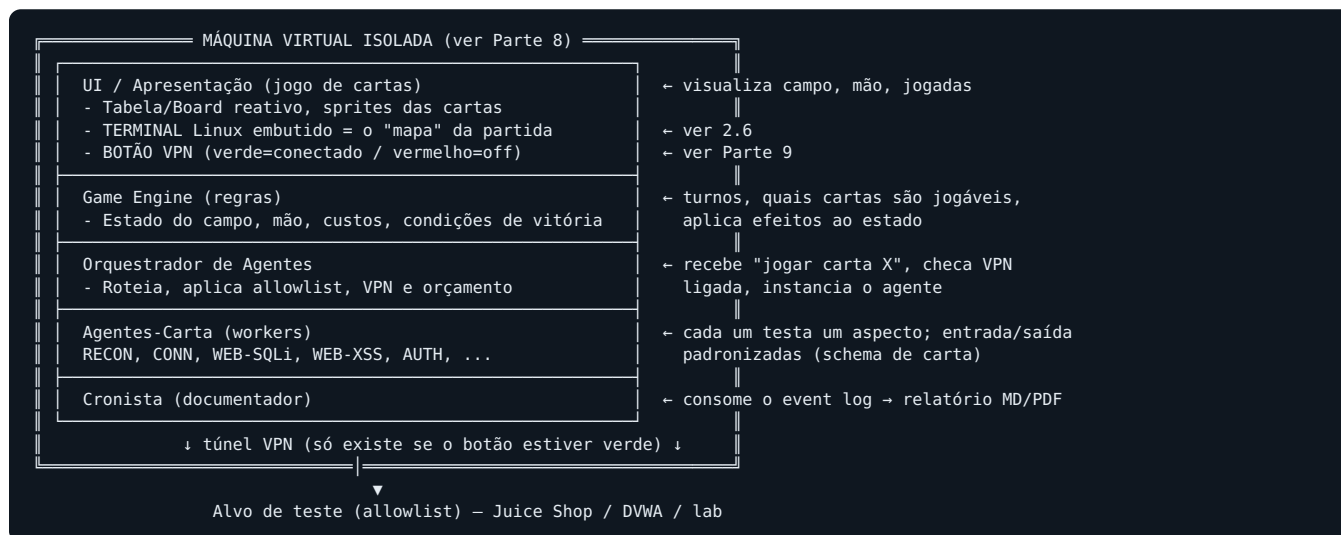
1.3 Por que isso é interessante para a IC (contribuição científica)

O ângulo de pesquisa não é "mais um scanner". É: 1. **Orquestração adaptativa por estado**: as cartas disponíveis emergem do que foi descoberto — um pipeline de pentest que se reconfigura sozinho, em vez de um playbook fixo. 2. **Cada capacidade encapsulada como agente-carta reutilizável e componível** (schema padronizado). 3. **Documentação automática e rastreável** como cidadão de primeira classe (reprodutibilidade). 4. A camada de jogo é a **interface de visualização/explicação** do fluxo — útil tanto para operar quanto para ensinar.

Métricas candidatas para o artigo: cobertura de superfície por nº de jogadas, custo (requisições) por achado, taxa de falso-positivo por tipo de carta, reprodutibilidade do relatório entre execuções, comparação "política de escolha de cartas" (aleatória vs. gulosa vs. LLM-guiada).

PARTE 2 — Arquitetura do sistema

2.1 Camadas



Note a moldura dupla: a partir daqui, **tudo** — engine, agentes, terminal, cronista — roda **dentro** de um ambiente isolado (VM/container), e a única saída para o alvo é um túnel VPN cujo estado é visível e controlável na UI. Isso é detalhado na Parte 9.

2.2 Stack sugerida (ajuste ao seu conforto)

- **Backend / orquestração:** Python 3.11+. Orquestrador simples próprio (recomendado para clareza no artigo) ou LangGraph se quiser grafo de estados pronto.
- **Agentes:** cada carta é uma classe Python que herda de uma base `CardAgent`. Ferramentas reais chamadas via subprocess/lib: `nmap` (python-nmap), `requests/httpx`, `sqlmap` (subprocess, alvo de lab), etc. Onde um LLM ajuda (ex: interpretar respostas HTTP, decidir payloads), chame a API Claude.

- **Estado / eventos:** um `event log` append-only em JSONL + um objeto `Field` em memória reconstruível a partir do log (event sourcing leve → dá reprodutibilidade de graça).
- **Frontend do jogo: Web** — React + Vite + TypeScript + Canvas/HTML. Comunicação com backend via WebSocket (para ver jogadas ao vivo) ou REST + polling.
- **Sprites das cartas:** gerados com **Codex** (ver Parte 5). Formato alvo: PNG/SVG por carta + um atlas.
- **Relatório:** Markdown → PDF (usar a skill/pipeline de PDF depois).

2.3 Esquema de dados — a Carta (contrato central)

Tudo gira em torno deste schema. Defina-o cedo e não quebre.

```
# card_schema (uma definição por agente-carta)
id: WEB-SQLi
name: "Injeção SQL"
category: exploitation          # recon | connectivity | enumeration | exploitation | post
arcana: mago                    # Arcano Maior que dá a arte/arquétipo (ver Parte 5.2)
badge: "SQLi"                  # rótulo curto sobreposto ao sprite do arcano
aspect: sql                     # o "aspecto" que casa com o estado do campo
cost: 3                         # mana/orçamento
preconditions:                  # o que precisa existir no campo para entrar na mão
  requires_state:
    - "web.forms_found > 0 OR web.params_found > 0"
inputs:                          # o que o agente lê do campo
  - target_url
  - discovered_params
effects:                          # como o resultado altera o campo (declarativo)
  reveals:
    - "vuln.sqli"
  may_unlock_cards:
    - AUTH-Bypass
    - DATA-Exfil
risk: high
llm_assisted: true              # usa Claude para escolher/adaptar payloads
config:                          # parâmetros pré-configurados por você
  payload_set: "generic-error-based"
  max_requests: 50
```

E o **resultado** padronizado que todo agente devolve:

```
{
  "card_id": "WEB-SQLi",
  "target": "http://localhost:3000/rest/products/search",
  "status": "success | no_finding | error | blocked_by_allowlist",
  "findings": [
    { "type": "sqli", "severity": "high", "evidence": "...", "location": "q param" }
  ],
  "state_delta": { "reveals": ["vuln.sqli"], "unlocks": ["DATA-Exfil"] },
  "requests_made": 37,
  "activity": [
    { "ts": "...", "phase": "exploit", "level": "info", "message": "sqlmap: parâmetro q injetável" }
  ],
  "artifacts": [
    { "type": "http_exchange", "ref": "art/req_resp_01.json", "meta": { "param": "q" } },
    { "type": "file", "ref": "art/dump_users.csv", "meta": { "rows": 128 } }
  ],
  "narrative": "Frase curta para o cronista e para a UI do jogo."
}
```

2.4 Regra de "quais cartas estão na mão"

O Game Engine avalia, a cada turno, as `preconditions` de cada carta contra o `Field`. As que passam e cabem no orçamento entram na **mão**. Isso é literalmente o mecanismo que o usuário descreveu: *"tenho na mão só as cartas que se encaixam no aspecto sobre o que nos deparamos na fase de teste"*. A seção 2.7 detalha **como** essas opções são calculadas e completadas automaticamente — é a peça que faltava fechar.

2.7 O Motor de Completação — como as opções aparecem no campo (peça central)

Esta é a mecânica que você pediu para "cuidar da completção": o sistema precisa **descobrir e oferecer automaticamente** as explorações possíveis, em vez de você adivinhar. É o que transforma uma lista estática de cartas num campo vivo.

O conceito-chave: carta-tipo vs. carta-opção (instância)

- Uma **carta-tipo** é a definição genérica em `config/cards/` (ex: "WEB-SQLi", que sabe testar SQL injection).
- Uma **carta-opção** (instância jogável) é uma carta-tipo **ligada a um alvo concreto já descoberto no campo** (ex: "WEB-SQLi → param q de /rest/products").

O campo nunca mostra carta-tipo solta; ele mostra **carta-opção** — cada exploração possível *sobre algo que existe agora no alvo*. É por isso que jogar uma carta muda o campo e, quase sempre, **faz nascer novas cartas-opção** (porque revelou novos alvos concretos).

Como o motor completa as opções (algoritmo)

A cada mudança no `Field`, o Game Engine roda `expand_options(field)`:

1. Enumere as ENTIDADES descobertas no campo:
hosts, portas abertas, tecnologias, endpoints, formulários, parâmetros, vulnerabilidades já reveladas, credenciais, etc.
2. Para cada carta-tipo no registry:
avalie suas preconditions contra o campo (2.3).
3. Para cada carta-tipo que passou, INSTANCIE uma carta-opção para CADA entidade compatível (ex: WEB-SQLi gera uma opção por parâmetro injetável descoberto).
4. Filtre pelo orçamento (mana) e por opções já jogadas (não re-oferecer o idêntico).
5. Ordene por relevância (risco/impacto/custo) — vira a ordem de exibição no campo.
6. Devolva a lista de cartas-opção = as "explorações possíveis agora".

Essa lista é o que a UI desenha no campo e o que o terminal-mapa autocompleta. **Completação** aqui tem dois sentidos, ambos implementados: - **Completação semântica**: o motor *completa o quadro* — deriva sozinho todas as jogadas possíveis a partir do estado, para você não ter que saber quais existem. - **Completação de entrada (tab-completion)**: no terminal-mapa, digitar `play + TAB` lista exatamente essas cartas-opção; digitar parte do nome completa. É a mesma lista, exposta como autocomplete de shell.

O ciclo de mutação (o que acontece ao jogar)

```
selecionar carta-opção → orquestrador checa allowlist + VPN → agente executa
→ AgentResult.state_delta é aplicado ao Field (novas entidades/vulns)
→ Engine roda expand_options(field) de novo
→ novas cartas-opção surgem no campo; algumas antigas somem (já exploradas)
→ cronista registra tudo no EventLog
```

Ponto de projeto importante: `expand_options` deve ser **puro e determinístico** dado o `Field` — assim o campo é sempre reconstruível a partir do EventLog (mantém a reprodutibilidade da Parte 2.5) e testável (dado um campo, as opções esperadas são fixas).

Completação assistida por LLM (opcional, mas forte para o artigo)

Além das opções derivadas por regra, um agente "sugeridor" pode usar o Claude para **propor explorações não óbvias** a partir do estado (ex: "essa combinação de header + tecnologia sugere testar CVE X"). Essas viram cartas-opção marcadas como *sugeridas por IA*, sempre passando pelas mesmas travas (allowlist/VPN) antes de executar. Isso conecta diretamente com o tema de agentes da sua IC: o pipeline não só executa — ele **raciocina sobre o que explorar em seguida**.

2.8 A base de conhecimento dos caminhos — os caminhos JÁ EXISTEM (attack graph)

Ponto crucial que você levantou: **os caminhos possíveis não são inventados por nós — eles já existem no mundo do pentest**. Existe um corpo de conhecimento consolidado que descreve exatamente quais técnicas existem, em que ordem (a *linearidade*) e o que cada uma habilita a seguir (a *ramificação*). O motor de completção (2.7) **não deve adivinhar** as relações entre cartas: ele deve **percorrer essa base de conhecimento real**.

As fontes reais (é daqui que os caminhos vêm)

- **MITRE ATT&CK** — a taxonomia de referência: *táticas* (o "porquê": Reconnaissance, Initial Access, Execution, Discovery, Lateral Movement, Exfiltration, etc.) organizadas em ordem, e *técnicas/sub-técnicas* dentro de cada tática (o "como"). Isso dá tanto a **linearidade** (ordem das táticas) quanto o vocabulário de nós.
- **PTES** (Penetration Testing Execution Standard) — as fases clássicas de um pentest (pré-engajamento, coleta de informações, modelagem de ameaças, análise de vulnerabilidades, exploração, pós-exploração, relatório). Mapeia direto nas suas "fases/turnos".

- **OWASP Testing Guide / Top 10** — o detalhamento para o mundo web (onde estão SQLi, XSS, auth, etc.), que é o foco de "pentest em links".
- **Cyber Kill Chain** — visão macro linear (recon → weaponization → delivery → exploitation → installation → C2 → actions).

Modelo: um grafo de ataque (DAG), não uma lista

A base de conhecimento é um **grafo dirigido** onde: - **Nó = uma técnica** = uma **carta-tipo** (ex: T-Recon-PortScan, T-Web-SQLi, T-Auth-CredBruteforce). Cada nó carrega seu ID de ATT&CK quando existir (ex: T1595 Active Scanning), sua tática/fase, e o "aspecto" que casa com o campo. - **Aresta = relação "habilita"** (enables): uma técnica só faz sentido depois que outra revelou a condição para ela. Ex: PortScan → ServiceEnum → WebRecon → SQLi. É aqui que mora a **ramificação**: um nó pode habilitar **vários** filhos (WebRecon habilita SQLi, XSS, e Auth-tests ao mesmo tempo), e cada filho, ao ser jogado, pode abrir novos ramos. - **Camadas = as táticas/fases em ordem = a linearidade**: o grafo flui das fases iniciais (recon) para as finais (pós-exploração), mas com múltiplos caminhos paralelos dentro e entre camadas.

Ou seja: **linearidade** = o eixo das fases (ATT&CK/PTES); **ramificação** = as arestas enables que um mesmo nó dispara. O "campo de batalha" é a região do grafo que já foi *desbloqueada* pelo que se descobriu até agora.

Como isso se conecta ao que já temos

- As preconditions e may_unlock_cards do schema da carta (2.3) **não são escritas à mão de forma arbitrária** — elas são **derivadas das arestas do attack graph**. A base de conhecimento é a fonte da verdade; os YAMLS das cartas são a projeção dela.
- O expand_options(field) (2.7) passa a ser, formalmente, uma **travessia do attack graph**: "dado o subconjunto de nós já satisfeitos pelo estado do campo, quais nós vizinhos (filhos via enables) tornam-se agora alcançáveis?" Cada nó alcançável, instanciado sobre as entidades concretas descobertas, é uma carta-opção.
- A **Forja de Cartas** (Parte 10.1) passa a ter contexto: forjar uma carta nova = **adicionar/posicionar um nó no attack graph** (com suas arestas de entrada/saída), não só escrever um YAML solto.

Onde isso vive no projeto

config/knowledge/attack_graph.yaml — o grafo declarativo (nós + arestas + tática/fase + IDs de ATT&CK). Um loader (engine/knowledge.py) o carrega e oferece consultas do tipo children_of(node), phase_of(node), reachable_from(satisfied_nodes). O attack graph é **dado**, não código — assim dá para citar a fonte de cada nó no artigo e evoluir a cobertura sem mexer no motor.

Para o artigo: este é um argumento forte de validade. Você não está inventando um jogo arbitrário — está **modelando um corpo de conhecimento reconhecido (ATT&CK/PTES/OWASP) como um grafo navegável**, e usando agentes para percorrê-lo adaptativamente. A "gamificação" é a interface; o grafo de ataque real é o substrato científico.

2.9 Catálogo inicial de cartas (a lista para começar a construir)

Regra de granularidade que você definiu: **uma carta por técnica que faz sentido como nó**, não uma carta por micro-variação. Uma carta-tipo cobre uma técnica; as micro-variações viram cartas-**opção** instanciadas em tempo real pelo motor de completção (2.7) — você não cria uma carta nova para cada endpoint, o motor faz isso sozinho.

Abaixo o **deck inicial** (foco em pentest web/links), já como nós do attack graph. Habilita = arestas de saída; Requer = o que precisa estar no campo (aresta de entrada). Construa nesta ordem — de cima para baixo é a linearidade das fases.

Carta-tipo	Fase (PTES)	Tática ATT&CK	Aspecto	Requer (entrada)	Habilita (saída)
RECON-PortScan	Coleta	Reconnaissance (T1595)	portas	alvo na allowlist	RECON-ServiceEnum
RECON-ServiceEnum	Coleta	Discovery (T1046)	serviços	porta aberta	CONN-TLS , WEB-Recon
CONN-TLS	Análise vuln.	Discovery	tls	serviço TLS/HTTPS	WEB-Recon
WEB-Recon	Coleta	Reconnaissance (T1595.003)	web	serviço HTTP(S)	WEB-DirEnum , WEB-TechFingerprint
WEB-TechFingerprint	Coleta	Discovery	web	resposta HTTP	WEB-KnownCVE
WEB-DirEnum	Coleta	Discovery	web	serviço HTTP(S)	WEB-SQLi , WEB-XSS , AUTH-LoginDiscovery
WEB-SQLi	Exploração	Initial Access / Injection (OWASP A03)	sql	parâmetro/form descoberto	DATA-Exfil , AUTH-Bypass
WEB-XSS	Exploração	Injection (OWASP A03)	xss	campo refletido/armazenado	SESSION-Hijack
AUTH-LoginDiscovery	Coleta	Discovery	auth	endpoint de login	AUTH-CredBruteforce , AUTH-DefaultCreds
AUTH-DefaultCreds	Exploração	Credential Access (T1110.004)	auth	tela de login	POST-SessionUse
AUTH-CredBruteforce	Exploração	Credential Access (T1110)	auth	tela de login	POST-SessionUse
AUTH-Bypass	Exploração	Initial Access	auth	SQLi confirmada em auth	POST-SessionUse
WEB-KnownCVE	Exploração	Exploit Public-Facing App (T1190)	cve	tecnologia+versão	(varia por CVE)
DATA-Exfil	Pós-exploração	Exfiltration (T1041)	dados	acesso a dados (via SQLi)	REPORT-Finding
SESSION-Hijack	Pós-exploração	Credential Access	sessão	XSS confirmada	POST-SessionUse
POST-SessionUse	Pós-exploração	Lateral Movement / Impact	pós	credencial/sessão válida	REPORT-Finding
REPORT-Finding	Relatório	—	meta	qualquer achado	(encerra ramo)

Sugestão de construção: comece por RECON-PortScan → RECON-ServiceEnum → WEB-Recon → WEB-DirEnum → WEB-SQLi / WEB-XSS (é o menor caminho jogável ponta-a-ponta contra o Juice Shop) e só depois preencha os ramos de AUTH e POST. Cada linha da tabela vira **um nó em attack_graph.yaml + um config/cards/<id>.yaml + um agents/<id>.py** — e é exatamente o que a Forja (10.1) automatiza depois.

A tabela acima é só o **deck mínimo** para ter algo jogável rápido. O grafo completo, com muitos caminhos, está na seção 2.10 — é o alvo real do projeto.

2.10 Grafo de ataque EXPANDIDO — muitos caminhos (o alvo real)

Aqui está o grafo grande. Ele é a tradução direta dos frameworks reais: **camadas = táticas do MITRE ATT&CK / fases do PTES (a linearidade)** e **arestas = relações enables (a ramificação)**. O ponto de ter muitos caminhos é que o campo quase nunca é linear: um mesmo achado (ex: "descobri um formulário") **abre vários ramos simultâneos** (SQLi, XSS, SSTI, CmdInjection...), e vários ramos **convergem** no mesmo objetivo (SQLi, força-bruta e JWT-attack todos podem levar a "sessão válida"). É essa topologia de grafo — não uma árvore, não uma lista — que o motor de completção (2.7) percorre.

Cada nó abaixo carrega, entre parênteses, sua âncora real: ATT&CK Txxxx (técnica) e/ou OWASP Axx:2025 (categoria). Isso é o que você cita no artigo.

Camada 1 — Reconhecimento (ATT&CK TA0043 · PTES Info Gathering)

Nó (carta-tipo)	Âncora	Requer	Habilita
RECON-PassiveDNS	ATT&CK T1596	alvo na allowlist	RECON-SubdomainEnum, RECON-CertTransparency
RECON-SubdomainEnum	T1595.003	domínio	RECON-PortScan, RECON-WebProbe
RECON-CertTransparency	T1596.001	domínio	RECON-SubdomainEnum, CRYPTO-TLSAnalysis
RECON-OSINT-Dorking	T1593	domínio	RECON-JSAnalysis, CRYPTO-ExposedSecrets
RECON-PortScan	T1595.001	host vivo	RECON-ServiceEnum
RECON-ServiceEnum	T1046	porta aberta	RECON-WebProbe, CRYPTO-TLSAnalysis, MISC-ServiceMisconfig
RECON-WebProbe	T1595.003	serviço HTTP(S)	RECON-TechFingerprint, RECON-DirEnum, RECON-WebCrawl, MISC-SecurityHeaders
RECON-TechFingerprint	T1592.002	resposta HTTP	SC-VulnComponent, INJ-SSTI, AUTH-JWTAttack
RECON-WebCrawl	T1595.003	app web	RECON-ParamDiscovery, AUTH-LoginDiscovery, INTEG-FileUploadDiscovery
RECON-DirEnum	— (content discovery)	app web	MISC-DirListing, MISC-ExposedGit, MISC-ExposedAdmin, MISC-BackupFiles
RECON-ParamDiscovery	—	endpoints	INJ-SQLi, INJ-CmdInjection, XSS-Reflected, BAC-IDOR, SSRF, BAC-PathTraversal, OPEN-Redirect
RECON-JSAnalysis	T1592	assets JS	RECON-ParamDiscovery, CRYPTO-ExposedSecrets, RECON-HiddenAPI
RECON-HiddenAPI	—	endpoints de API	BAC-IDOR, BAC-MassAssignment, AUTH-JWTAttack

Camada 2 — Análise de vulnerabilidades / Weaponization (PTES Vuln Analysis)

Nó	Âncora	Requer	Habilita
CRYPTO-TLSAnalysis	OWASP A04:2025	serviço TLS	CRYPTO-WeakCipher, CRYPTO-DataInTransit
CRYPTO-ExposedSecrets	A04:2025	JS/dorking	AUTH-JWTAttack, SC-APIKeyAbuse
MISC-SecurityHeaders	A02:2025	resposta HTTP	MISC-CORSCheck, XSS-Reflected, INTEG-CSRF
MISC-CORSCheck	A02:2025	headers	BAC-CrossOrigin
SC-VulnComponent	A03:2025 · T1595.002	tecnologia+versão	SC-KnownCVE-Exploit
MISC-ServiceMisconfig	A02:2025	serviço enumerado	MISC-DefaultCreds, SC-KnownCVE-Exploit

Camada 3 — Exploração / Initial Access (o grande leque de ramos)

Injeção (OWASP A05:2025 · ATT&CK T1190/T1059)

Nó	Âncora	Requer	Habilita
INJ-SQLi	A05:2025 · T1190	parâmetro injetável	POST-DataExfil, AUTH-Bypass, INJ-SQLi-RCE
INJ-NoSQLi	A05:2025	param em app NoSQL	POST-DataExfil, AUTH-Bypass
INJ-CmdInjection	A05:2025 · T1059	param que toca SO	POST-RCE, POST-ReverseShell
INJ-SSTI	A05:2025	template engine detectada	POST-RCE
INJ-XXE	A05:2025	endpoint que parseia XML	SSRF, BAC-PathTraversal, POST-DataExfil
INJ-LDAPi	A05:2025	campo que consulta LDAP	AUTH-Bypass
INJ-HeaderCRLF	A05:2025	header refletido	XSS-Reflected, OPEN-Redirect
XSS-Reflected	A05:2025	input refletido	POST-SessionHijack, INTEG-CSRF
XSS-Stored	A05:2025	input persistido+exibido	POST-SessionHijack, POST-CredHarvest
XSS-DOM	A05:2025	sink JS client-side	POST-SessionHijack

Broken Access Control + SSRF (OWASP A01:2025)

Nó	Âncora	Requer	Habilita
BAC-IDOR	A01:2025	ID de objeto em request	POST-DataExfil , BAC-PrivEsc
BAC-PathTraversal	A01:2025	param de caminho/arquivo	POST-DataExfil , MISC-BackupFiles , POST-RCE
BAC-PrivEsc	A01:2025	acesso autenticado baixo	POST-AdminAccess
BAC-MissingFunctionAuth	A01:2025	função sensível sem checagem	POST-AdminAccess
BAC-MassAssignment	A01:2025	API que aceita objeto	BAC-PrivEsc
SSRF	A01:2025 (SSRF)	param que faz request server-side	RECON-InternalScan , CRYPTO-ExposedSecrets , POST-CloudMetadata
BAC-CrossOrigin	A01:2025	CORS permissivo	POST-DataExfil

Falhas de autenticação (OWASP A07:2025 · ATT&CK T1110)

Nó	Âncora	Requer	Habilita
AUTH-LoginDiscovery	A07:2025	endpoint de login	AUTH-DefaultCreds , AUTH-CredBruteforce , AUTH-CredStuffing , AUTH-WeakReset
AUTH-DefaultCreds	A07:2025 · T1110.004	tela de login	POST-SessionUse
AUTH-CredBruteforce	T1110.001	tela de login	POST-SessionUse
AUTH-CredStuffing	T1110.004	login + lista vazada	POST-SessionUse
AUTH-WeakReset	A07:2025	fluxo de reset de senha	POST-AccountTakeover
AUTH-JWTAttack	A07:2025	token JWT	AUTH-Bypass , BAC-PrivEsc
AUTH-SessionFixation	A07:2025	sessão manipulável	POST-SessionHijack
AUTH-Bypass	A07:2025	SQLi/LDAPi/JWT em auth	POST-SessionUse

Misconfig / Supply chain / Design / Integridade / Exceções

Nó	Âncora	Requer	Habilita
MISC-DirListing	A02:2025	dir sem index	MISC-BackupFiles , CRYPTO-ExposedSecrets
MISC-ExposedGit	A02:2025	.git acessível	POST-SourceLeak , CRYPTO-ExposedSecrets
MISC-ExposedAdmin	A02:2025	painel admin exposto	AUTH-DefaultCreds , BAC-MissingFunctionAuth
MISC-BackupFiles	A02:2025	arquivos .bak/.zip	POST-SourceLeak
MISC-DefaultCreds	A02:2025	serviço com login padrão	POST-SessionUse
SC-KnownCVE-Exploit	A03:2025 · T1190	componente vulnerável	POST-RCE
SC-SubdomainTakeover	A03:2025	CNAME órfão	POST-Phishing-Infra
SC-APIKeyAbuse	A03:2025	chave exposta	POST-DataExfil
DESIGN-BusinessLogic	A06:2025	fluxo multi-etapa	POST-Fraud
DESIGN-RaceCondition	A06:2025	operação concorrente	POST-Fraud
INTEG-InsecureDeser	A08:2025 · T1190	objeto serializado no input	POST-RCE
INTEG-CSRF	A08:2025	ação sem anti-CSRF	POST-AccountTakeover
INTEG-FileUpload	A08:2025	upload sem validação	POST-RCE
EXC-VerboseErrors	A10:2025	erro detalhado vazado	INJ-SQLi , CRYPTO-ExposedSecrets
EXC-OpenRedirect / OPEN-Redirect	A10:2025	param de redirect	POST-Phishing , SSRF
LOG-DetectionCheck	A09:2025	qualquer ação ofensiva	(meta: mede se o alvo detecta/loga)

Camada 4 — Pós-exploração (ATT&CK TA0006/07/08/09/10/40)

Nó	Âncora	Requer	Habilita
POST-SessionUse	T1078	credencial/sessão válida	BAC-PrivEsc, POST-DataExfil, POST-LateralMovement
POST-SessionHijack	T1539	cookie/token roubado (XSS)	POST-AccountTakeover
POST-AccountTakeover	T1586	sessão/reset de vítima	POST-DataExfil, POST-Fraud
POST-RCE	T1190/T1059	execução de código	POST-ReverseShell, POST-Persistence, POST-Pivot
POST-ReverseShell	T1059	RCE	POST-Pivot, POST-CredDump
POST-CredDump	TA0006	shell/acesso	POST-LateralMovement
POST-DataExfil	T1041	acesso a dados	REPORT-Finding
POST-InternalScan / RECON-InternalScan	T1046	SSRF/pivot	POST-LateralMovement, POST-CloudMetadata
POST-CloudMetadata	T1552.005	SSRF p/ 169.254.169.254	SC-APIKeyAbuse, POST-LateralMovement
POST-LateralMovement	TA0008	acesso interno	POST-Persistence, POST-DataExfil
POST-Persistence	TA0003	acesso estabelecido	REPORT-Finding
VALIDATE-Finding	— (A Justiça, 5.5)	qualquer achado bruto	REPORT-Finding (só se confirmed)
REPORT-Finding	— (PTES Reporting)	achado validado	(encerra o ramo, alimenta o cronista)

Padrões de topologia que o grafo captura (por que "muitos caminhos")

- **Divergência (um achado abre N ramos):** RECON-ParamDiscovery sozinho habilita SQLi, CmdInjection, XSS, IDOR, SSRF, PathTraversal e OpenRedirect — sete ramos paralelos de um único nó.
- **Convergência (N ramos, um objetivo):** INJ-SQLi, AUTH-CredBruteforce, AUTH-JWTAttack e AUTH-Bypass todos levam a POST-SessionUse — o mesmo objetivo por caminhos diferentes. Ótimo para o artigo comparar qual política de escolha chega mais barato.
- **Ciclos de aprofundamento:** SSRF → RECON-InternalScan → POST-LateralMovement → POST-DataExfil mostra recon acontecendo *de novo*, agora por dentro — o grafo permite reentrar em fases anteriores num contexto novo.
- **Nós-meta transversais:** LOG-DetectionCheck (A09) e EXC-VerboseErrors (A10) não são um "caminho" clássico — cruzam o grafo inteiro medindo defesa/vazamento, e podem ser jogados a qualquer momento.

Como isso fica em attack_graph.yaml (formato)

```
nodes:
  RECON-ParamDiscovery:
    attack: null
    owasp: null
    phase: recon
    aspect: web
    enables: [INJ-SQLi, INJ-CmdInjection, XSS-Reflected, BAC-IDOR, SSRF, BAC-PathTraversal, OPEN-Redirect]
  INJ-SQLi:
    attack: T1190
    owasp: "A05:2025"
    phase: exploitation
    aspect: sql
    requires: [RECON-ParamDiscovery] # aresta de entrada (derivada dos 'enables' dos pais)
    enables: [POST-DataExfil, AUTH-Bypass, INJ-SQLi-RCE]
    risk: high
  POST-SessionUse:
    attack: T1078
    phase: post
    aspect: post
    requires: [AUTH-DefaultCreds, AUTH-CredBruteforce, AUTH-JWTAttack, AUTH-Bypass] # convergência
    enables: [BAC-PrivEsc, POST-DataExfil, POST-LateralMovement]
```

O loader (engine/knowledge.py) valida que o grafo é um DAG coerente (sem aresta órfã, todo requires aponta para um nó existente) e expõe reachable_from(satisfied_nodes) — que é o que expand_options (2.7) usa para revelar as próximas cartas-opção.

Escala e roadmap de construção: não construa os ~60 nós de uma vez. Modele o grafo inteiro em `attack_graph.yaml` desde já (é só dado), mas implemente os **agentes** em ondas: (1) linha RECON-SQLi/XSS , (2) ramo AUTH , (3) ramos BAC/SSRF , (4) INJ avançada + SC/CVE , (5) POST . Cada onda é uma leva de cartas jogáveis nova, e um marco mensurável para o artigo.

2.11 Mais frameworks, mais categorias — API e LLM (o grafo fica robusto)

Para o grafo ficar realmente bom e defensável, adicionamos dois frameworks oficiais além do ATT&CK e do OWASP Top 10:2025 web. Ambos são padrões reconhecidos — de novo, os caminhos **já existem no mundo**, só estamos modelando-os.

OWASP API Security Top 10 (2023) — camada de API

APIs têm seu próprio Top 10, porque "pentest em links" hoje é, em grande parte, atacar endpoints de API. Novos nós (âncora `API...:2023`):

Nó	Âncora	Requer	Habilita
API-SchemaDiscovery	API9:2023 (Improper Inventory)	endpoint de API / Swagger / OpenAPI	API-BOLA , API-BFLA , API-B0PLA , GraphQL-Introspection
API-BOLA	API1:2023 (Broken Object Level Auth)	ID de objeto em request de API	POST-DataExfil , BAC-PrivEsc
API-BrokenAuth	API2:2023	endpoint de auth de API	AUTH-JWTAttack , POST-SessionUse
API-B0PLA	API3:2023 (Object Property Level Auth)	objeto com props sensíveis	BAC-MassAssignment , POST-DataExfil
API-ResourceConsumption	API4:2023	endpoint sem rate-limit ([!] só lab)	LLM-UnboundedConsumption , DESIGN-RaceCondition
API-BFLA	API5:2023 (Function Level Auth)	função admin via API	POST-AdminAccess
API-BusinessFlowAbuse	API6:2023	fluxo de negócio exposto	POST-Fraud
API-SSRF	API7:2023	param que faz fetch server-side	SSRF (converge)
API-UnsafeConsumption	API10:2023	API consome API terceira	SC-APIKeyAbuse
GraphQL-Introspection	API9:2023	endpoint GraphQL	GraphQL-Injection , GraphQL-BatchingAbuse , API-BOLA
GraphQL-Injection	A05:2025	query GraphQL manipulável	POST-DataExfil
GraphQL-BatchingAbuse	API4:2023	batching habilitado	AUTH-CredBruteforce (converge)

OWASP Top 10 for LLM Applications (2025) — camada de IA (conecta com sua IC!)

Este é o diferencial da sua pesquisa: se o alvo tem um chatbot/feature com LLM (cada vez mais comum), há uma superfície inteira de ataque com framework próprio. E, meta-nível, é a mesma classe de risco que **os seus próprios agentes** enfrentam. Novos nós (âncora `LLM...:2025`):

Nó	Âncora	Requer	Habilita
LLM-Discovery	—	feature/chatbot com LLM detectado	LLM-PromptInjection , LLM-SystemPromptLeak
LLM-PromptInjection	LLM01:2025 (Prompt Injection)	entrada que chega ao prompt	LLM-SensitiveDisclosure , LLM-ExcessiveAgency , LLM-InsecureOutput
LLM-SensitiveDisclosure	LLM02:2025	LLM com acesso a dados	POST-DataExfil
LLM-SystemPromptLeak	LLM07:2025	LLM responsivo a sondagem	LLM-PromptInjection (aprofunda)
LLM-InsecureOutput	LLM05:2025 (Improper Output Handling)	saída do LLM renderizada/executada	XSS-Stored , POST-RCE (ponte p/ web!)
LLM-ExcessiveAgency	LLM06:2025	LLM com ferramentas/plugins	POST-RCE , API-BFLA , SSRF
LLM-VectorEmbedding	LLM08:2025	app com RAG/embeddings	LLM-SensitiveDisclosure
LLM-UnboundedConsumption	LLM10:2025	LLM sem limite de uso	(meta: custo/DoS, só lab)

Observação para o artigo: a categoria LLM cria um **loop reflexivo** interessante — seu pipeline de agentes testa vulnerabilidades de *outras* aplicações de IA usando as mesmas técnicas (prompt injection, excessive agency) que ameaçam o próprio pipeline. Isso é material rico para a seção de discussão.

Mais nós de protocolo/web avançado (completando o leque)

Nó	Âncora	Requer	Habilita
RECON-WAFDetect	—	serviço HTTP(S)	EVASION-WAFBypass
RECON-RobotsSitemap	—	app web	RECON-DirEnum (converge)
RECON-VHostEnum	T1595.003	IP com múltiplos hosts	RECON-WebProbe (converge)
HTTP-RequestSmuggling	A02:2025	front/back-end HTTP distintos	POST-CacheDeception , AUTH-Bypass
CACHE-Poisoning	A02:2025	cache compartilhado + input em cache	POST-Phishing , XSS-Stored
WS-WebSocketHijack	A01:2025	WebSocket sem origem validada	POST-SessionHijack
CLICKJACK	A02:2025	sem X-Frame-Options/CSP frame-ancestors	INTEG-CSRF (converge)
OAuth-Attack	A07:2025	fluxo OAuth/SSO	POST-AccountTakeover
MFA-Bypass	A07:2025	2FA presente	POST-AccountTakeover
EVASION-WAFBypass	— (transversal)	WAF detectado	(re-habilita nós de INJ com payloads evasivos)

Com isso o grafo passa de ~60 para **~90 nós**, cobrindo web, API, LLM e protocolo — um substrato científico sólido. Continua sendo só **dado** em `attack_graph.yaml` ; os agentes vêm em ondas.

2.12 O Agente-Operador de Ferramenta (o agente que "gere uma ferramenta")

Você pediu explicitamente: um agente **com capacidade de gerir uma ferramenta** para resolver o problema de cada nó. Este é o padrão que torna cada carta poderosa sem reescrever lógica de rede à mão — o agente não é a ferramenta, ele **opera** a ferramenta certa.

O conceito

Cada nó do attack graph declara qual(is) ferramenta(s) resolve(m) aquele problema. O agente-carta herda de um **ToolOperatorAgent** , cujo trabalho é um *loop de operação de ferramenta* assistido por LLM:

1. SELECIONAR a ferramenta para este nó (do config/tools.yaml).
2. CONSTRUIR a invocação: montar argumentos/payloads a partir do estado do campo (target, params, endpoints descobertos) + config da carta. (LLM ajuda a adaptar.)
3. CHECAR travas: allowlist + VPN ligada + timeout + teto de requisições.
4. EXECUTAR a ferramenta (subprocess, dentro da VM Linux) capturando stdout/stderr/exit.
5. INTERPRETAR a saída → findings estruturados (parser dedicado + LLM p/ casos ambíguos).
6. DECIDIR follow-up: a saída sugere uma segunda invocação mais precisa? (ex: sqlmap confirmou injeção → rodar de novo para extrair). Repete até resolver ou esgotar orçamento.
7. DEVOLVER AgentResult (schema 2.3) com findings + state_delta + narrativa p/ o cronista.

O que muda entre um agente e outro é basicamente **qual ferramenta** e **como parsear a saída** — a lógica de operação (travas, timeout, loop, logging) fica toda na base. Isso é o que permite forjar cartas novas rapidamente.

Registro de ferramentas — config/tools.yaml

Mapeia técnica → ferramenta(s). Exemplo (recorte):

```
tools:
  nmap: { bin: nmap, kind: recon, net: true }
  httpx: { bin: httpx, kind: recon, net: true }
  ffuf: { bin: ffuf, kind: content, net: true }
  nuclei: { bin: nuclei, kind: vuln, net: true }
  sslscan: { bin: sslscan, kind: crypto, net: true }
  sqlmap: { bin: sqlmap, kind: exploit, net: true }
  dalfox: { bin: dalfox, kind: exploit, net: true } # XSS
  jwt_tool: { bin: jwt_tool, kind: exploit, net: false }
  hydra: { bin: hydra, kind: exploit, net: true }
  graphw00f: { bin: graphw00f, kind: recon, net: true } # GraphQL fingerprint
  garak: { bin: garak, kind: llm, net: true } # LLM red-team / prompt injection
# ... arsenal completo (2.18): rustscan, masscan, amass, subfinder, theharvester, dnsrecon,
# whatweb, gobuster, feroxbuster, dirsearch, nikto, wpscan, xssstrike, testssl, sslyze, arjun,
# newman, metasploit, searchsploit, netcat, socat, tshark, tcpdump, bettercap, proxychains,
# chisel, hashcat, john, impacket, netexec, bloodhound, responder, mimikatz, enum4linux-ng,
# peass, wafw00f, zap, burp – todas instaladas na attack-box, cada uma definida UMA vez aqui.

node_tools: # técnica (nó) → ferramenta(s) que a resolvem
  RECON-PortScan: [nmap]
  RECON-WebProbe: [httpx]
  RECON-DirEnum: [ffuf]
  RECON-TechFingerprint: [nuclei, httpx]
  CRYPTO-TLSAnalysis: [sslscan]
  SC-KnownCVE-Exploit: [nuclei]
  INJ-SQLi: [sqlmap]
  XSS-Reflected: [dalfox]
  AUTH-JWTAttack: [jwt_tool]
  AUTH-CredBruteforce: [hydra]
  GRAPHQL-Introspection: [graphw00f]
  LLM-PromptInjection: [garak]
```

Ferramentas com `net: true` só podem rodar com a **VPN ligada** (Parte 8) — a checagem já é feita pela base antes de qualquer execução. Ferramentas ausentes no sistema fazem o agente devolver `status: "tool_missing"` (nunca falha silenciosa).

2.13 Molde pronto de implementação de um agente-carta

Este é o "pré-moldado" que você pediu. Todo agente novo é uma subclasse curta — a metadata (âncora ATT&CK/OWASP), a ferramenta e o parser são o que você preenche. É também o `forge/templates/agent.py.j2` que a Forja (10.1) gera.

```

from dataclasses import dataclass, field
from typing import Literal
import subprocess, shlex

@dataclass
class AgentResult:
    # contrato da seção 2.3
    card_id: str
    target: str
    status: Literal["success", "no_finding", "error", "blocked_by_allowlist",
                  "blocked_by_vpn_down", "tool_missing"]
    findings: list = field(default_factory=list)
    state_delta: dict = field(default_factory=dict)
    requests_made: int = 0
    artifacts: list = field(default_factory=list)
    narrative: str = ""

class CardAgent:
    # --- metadata (âncora real, obrigatória) ---
    id: str # ex: "INJ-SQLi"
    attack_id: str | None # ex: "T1190" (MITRE ATT&CK)
    owasp_id: str | None # ex: "A05:2025" (OWASP)
    phase: str # ex: "exploitation"
    aspect: str # ex: "sql"

    def preconditions(self, field) -> bool: ... # derivadas do attack_graph (2.8)
    def build_options(self, field) -> list: ... # instancia cartas-opção (2.7)
    def run(self, field, option) -> AgentResult: ...

class ToolOperatorAgent(CardAgent):
    tools: list[str] = [] # do config/tools.yaml (2.12)

    def run(self, field, option) -> AgentResult:
        if not self.allowlist_ok(option.target):
            return AgentResult(self.id, option.target, "blocked_by_allowlist")
        if self.needs_net() and not self.vpn_up():
            return AgentResult(self.id, option.target, "blocked_by_vpn_down")
        tool = self.select_tool() # 1. seleciona
        if not tool.available():
            return AgentResult(self.id, option.target, "tool_missing")
        cmd = self.build_command(tool, field, option) # 2. constrói (LLM-assist opcional)
        raw = self.exec(cmd, timeout=self.timeout) # 3-4. trava + executa (subprocess na VM)
        findings = self.parse(raw) # 5. interpreta saída → findings
        delta = self.to_state_delta(findings) # efeitos no campo
        return AgentResult(self.id, option.target,
                          "success" if findings else "no_finding",
                          findings=findings, state_delta=delta,
                          requests_made=self.count, artifacts=[raw.log_path],
                          narrative=self.narrate(findings))

# allowlist_ok / vpn_up / exec / select_tool ficam implementados aqui na base

```

Exemplo preenchido — agents/inj_sqli.py

```
from agents.base import ToolOperatorAgent, AgentResult

class SqliAgent(ToolOperatorAgent):
    id = "INJ-SQLi"
    attack_id = "T1190" # MITRE ATT&CK: Exploit Public-Facing Application
    owasp_id = "A05:2025" # OWASP Top 10:2025 — Injection
    phase = "exploitation"
    aspect = "sql"
    tools = ["sqlmap"]

    def preconditions(self, field) -> bool:
        # derivado do attack_graph: INJ-SQLi requires RECON-ParamDiscovery
        return field.has("web.params_found") or field.has("web.forms_found")

    def build_options(self, field):
        # uma carta-opção por parâmetro injetável descoberto (motor de completção 2.7)
        return [self.option(target=p.url, param=p.name) for p in field.injectable_params()]

    def build_command(self, tool, field, option):
        return (f"sqlmap -u {option.target} -p {option.param} "
                f"--batch --level={self.config['level']} --risk={self.config['risk']} "
                f"--technique={self.config['payload_set']}")

    def parse(self, raw):
        # parser dedicado do output do sqlmap; LLM só para casos ambíguos
        return parse_sqlmap(raw.stdout) # -> [{type:'sqli', severity:'high', evidence:..., location:'q'}]

    def to_state_delta(self, findings):
        if not findings: return {}
        return {"reveals": ["vuln.sqli"], "unlocks": ["POST-DataExfil", "AUTH-Bypass"]}
```

Padrão para forjar qualquer carta nova: copie a **pasta-carta** de exemplo (2.17), troque `id / attack_id / owasp_id / phase / aspect` no `card.yaml`, aponte `tools`, escreva `build_command` e `parse` no `agent.py`, ajuste o `card.md`, e declare os `unlocks` conforme as arestas do attack graph. É deliberadamente pequeno — o poder está na base + no registro de ferramentas + na VM que já tem tudo instalado.

2.14 Catálogo mestre consolidado — o deck pré-configurado (sem repetições)

Organizei todas as técnicas que você listou (havia várias repetidas entre listas web/API/tendências-2026) em **um único catálogo canônico, deduplicado**, agrupado por família e já mapeado no arcano de tarô que dá a arte. **Este é o deck oficial** — supera as listas espalhadas das seções 2.9–2.11, que passam a ser vistas como rascunho. Coluna ★ = os 10 núcleos priorizados; **Âncora** = ATT&CK / OWASP (Web:2025, API:2023, LLM:2025); **Ferramenta** = o que o `ToolOperatorAgent` opera.

Família A — Injeção

#	Técnica	Arcano	Âncora	Ferramenta	★
A1	SQL Injection	O Mago	A05:2025 · T1190	sqlmap	★
A2	Blind SQL Injection (boolean/time)	O Mago	A05:2025	sqlmap	
A3	NoSQL Injection	O Mago	A05:2025	nosqli/custom	
A4	OS Command Injection	A Torre	A05:2025 · T1059	commix	★
A5	Code Injection	A Torre	A05:2025 · T1059	custom	
A6	Server-Side Template Injection (SSTI)	A Torre	A05:2025	tplmap	
A7	LDAP Injection	O Mago	A05:2025	custom	
A8	XPath Injection	O Mago	A05:2025	custom	
A9	XML External Entity (XXE)	O Mago	A05:2025	custom/Burp	
A10	CRLF / HTTP Header Injection	O Mago	A05:2025	custom	

Família B — Navegador e cliente

#	Técnica	Arcano	Âncora	Ferramenta	★
B1	Reflected XSS	O Mago	A05:2025	dalfox	★
B2	Stored XSS	O Mago	A05:2025	dalfox	★
B3	DOM-based XSS	O Mago	A05:2025	dalfox	
B4	HTML Injection	O Mago	A05:2025	custom	
B5	CSS Injection	O Mago	A05:2025	custom	
B6	CSRF	O Hierofante	A08:2025	Burp/custom	★
B7	Clickjacking	O Hierofante	A02:2025	custom	
B8	Open Redirect	Os Enamorados	A10:2025	custom	
B9	WebSocket Hijacking	O Diabo	A01:2025	custom	
B10	Prototype Pollution	A Torre	A05:2025	ppfuzz/custom	

Família C — Autenticação e credenciais

#	Técnica	Arcano	Âncora	Ferramenta	★
C1	Brute Force	A Força	A07:2025 · T1110.001	hydra/ffuf	★
C2	Credential Stuffing	A Força	T1110.004	custom	★
C3	Password Spraying	A Força	T1110.003	custom	
C4	Username Enumeration	A Força	A07:2025	ffuf	
C5	Authentication Bypass	A Força	A07:2025	custom	★
C6	MFA Bypass	A Força	A07:2025	custom	
C7	Password Reset Poisoning	A Força	A07:2025	custom (Host header)	
C8	OAuth/OIDC Misconfig Abuse	Os Enamorados	A07:2025	custom	
C9	JWT Attacks	Os Enamorados	A07:2025	jwt_tool	
C10	Default / Weak Credentials	A Força	A02:2025 · T1110.004	custom	

Família D — Sessão e autorização

#	Técnica	Arcano	Âncora	Ferramenta	★
D1	Session Hijacking	O Diabo	A07:2025 · T1539	custom	
D2	Session Fixation	O Diabo	A07:2025	custom	
D3	Insecure Cookie Abuse	O Diabo	A02:2025	custom	
D4	IDOR / BOLA	O Imperador	A01:2025 · API1:2023	custom	★
D5	Broken Function-Level Authorization	O Imperador	A01:2025 · API5:2023	custom	★
D6	Horizontal Privilege Escalation	O Imperador	A01:2025	custom	
D7	Vertical Privilege Escalation	O Imperador	A01:2025	custom	
D8	Path Traversal	A Sacerdotisa	A01:2025	ffuf/custom	★
D9	Local File Inclusion (LFI)	A Sacerdotisa	A01:2025	custom	
D10	Remote File Inclusion (RFI)	A Sacerdotisa	A01:2025	custom	

Família E — Servidor, infraestrutura e lógica

#	Técnica	Arcano	Âncora	Ferramenta	★
E1	Server-Side Request Forgery (SSRF)	O Carro	A01:2025 · API7:2023	custom	★
E2	Insecure File Upload	A Torre	A08:2025	custom	★
E3	Unrestricted File Download	A Sacerdotisa	A01:2025	custom	
E4	Insecure Deserialization	A Torre	A08:2025 · T1190	ysoserial/custom	
E5	HTTP Request Smuggling	A Temperança	A02:2025	smuggler	
E6	Host Header Injection	A Temperança	A02:2025	custom	
E7	CORS Misconfiguration	O Hierofante	A02:2025	custom	
E8	Mass Assignment / BOPLA	O Imperador	API3:2023	custom	
E9	Race Condition / TOCTOU	A Temperança	A06:2025	Turbo Intruder	
E10	Denial of Service / Resource Exhaustion ([!] só lab)	A Torre	API4:2023 · LLM10:2025	custom	

Família F — API (OWASP API Security Top 10:2023)

#	Técnica	Arcano	Âncora	Ferramenta	★
F1	Broken Authentication (API)	A Força	API2:2023	custom	
F2	Business Flow Abuse	A Roda da Fortuna	API6:2023	custom	
F3	Shadow / Zombie APIs	A Imperatriz	API9:2023	kiterunner	
F4	Unsafe API Consumption	Os Enamorados	API10:2023	custom	
F5	GraphQL Abuse (introspection/injection/batching)	A Imperatriz	API9:2023	graphw00f/InQL	

Família G — Aplicações com LLM (OWASP LLM:2025 · MITRE ATLAS)

#	Técnica	Arcano	Âncora	Ferramenta	★
G1	Prompt Injection (direto/indireto)	A Lua	LLM01:2025 · ATLAS	garak	
G2	System Prompt Leakage	A Lua	LLM07:2025 · ATLAS	garak/custom	
G3	Insecure Output Handling (ponte → XSS/RCE)	A Lua	LLM05:2025	custom	
G4	Excessive Agency (abuso de ferramentas do agente)	A Lua	LLM06:2025 · ATLAS	custom	
G5	Sensitive Info Disclosure / RAG leakage	A Lua	LLM02:2025 · ATLAS	custom	

Família H — Reconhecimento e descoberta (as raízes da árvore)

#	Técnica	Arcano	Âncora	Ferramenta	★
H1	Recon passivo inicial (semente)	O Louco	T1596	dnsx/passivo	
H2	OSINT / DNS / Subdomínios	O Eremita	T1595.003	subfinder/amass	
H3	Certificate Transparency	A Estrela	T1596.001	crt.sh/custom	
H4	Web Crawl / endpoints / params	A Imperatriz	T1595.003	katana/httpx	★
H5	Segredos ocultos (JS, .git, backups, metadados)	A Sacerdotisa	T1592	trufflehog/custom	
H6	Fingerprint de tecnologia + CVE conhecida	A Roda da Fortuna	T1592.002 · A06:2025	nuclei/whatweb	★
H7	Consolidação da superfície (visão)	O Sol	—	(engine)	
H8	WAF detect + evasão	O Enforcado	—	wafw00f/custom	

Família I — Validação, transição e fecho (cartas-sistema, ver 5.5)

#	Técnica	Arcano	Função
I1	Validação de achado	A Justiça	confirma/severidade/falso-positivo/detecção
I2	Transição de fase	A Morte	encerra ramo, promove à próxima fase
I3	Achado formal	O Julgamento	evidência+impacto+severidade+referência+correção
I4	Relatório final	O Mundo	aciona o Cronista, cobertura e custo, fecha a partida

Família J — Nuvem, containers e Kubernetes (ramo alcançado via POST-CloudMetadata)

#	Técnica	Arcano	Âncora	Ferramenta
J1	Auditoria de IAM / privilégios em nuvem	O Imperador	ATLAS/CIS	Prowler
J2	Auditoria de configuração de nuvem	O Hierofante	CIS Benchmarks	ScoutSuite
J3	CVE em imagem/container	A Roda da Fortuna	A06:2025	Trivy
J4	Ataque a cluster Kubernetes	O Carro	ATLAS	kube-hunter

Total: ~61 técnicas + 8 cartas-sistema, cobertas por **22 arcanos** de tarô, operando ~100 ferramentas. Os 10 núcleos (★) são o caminho crítico que deve funcionar primeiro contra o Juice Shop.

2.15 Deck e gameplay PRÉ-CONFIGURADOS

Você pediu que os agentes-carta e o gameplay já venham **prontos**, não configurados em tempo de jogo. Então o repositório nasce com o deck inteiro preenchido:

- **Cada** `config/cards/<id>.yaml` **já vem com tudo definido** — arcano, badge, ferramenta, `payload_set`, limites (`max_requests`, `timeout`), risco, preconditions e efeitos derivados do grafo. O jogador não configura nada para jogar; a Forja (10.1) é só para *criar cartas novas*, não para operar as existentes. Exemplo pré-preenchido:

```
id: E1-SSRF
arcana: carro
badge: "SSRF"
category: exploitation
aspect: ssrf
attack: "T1190"; owasp: "A01:2025" # âncora real (obrigatória)
tool: custom-ssrf
preconditions: { requires_state: ["web.params_found > 0"] }
config: { callback: "burp-collaborator-lab", max_requests: 40, timeout_s: 90, risk: high }
effects: { reveals: ["vuln.ssrff"], may_unlock_cards: ["Carro-InternalScan", "Carro-CloudMetadata"] }
```

- **O gameplay já vem calibrado** em `config/game_rules.yaml`: mão inicial = só **O Louco** (recon passivo); economia de mana por família (recon barato, exploração cara, pós-exploração caríssima); objetivo padrão e condição de derrota; e a ordem das fases (a linearidade da árvore). Tudo editável, mas funcional out-of-the-box.
- `config/arcana.yaml` **já traz os 22 arcanos** com número, nome, tipo (técnica/sistema) e a paleta/temática do sprite. Assim a UI e a Forja sempre sabem a qual arcano uma técnica pertence.

2.16 A árvore NÃO-DETERMINÍSTICA — como novos cenários emergem durante a partida

Este é o comportamento que você descreveu: a "árvore" de caminhos **não é fixa** — ela se **cria conforme se joga**, gerando cenários novos a cada partida. Formalizando:

O attack graph (2.8) é o **espaço de possibilidades** (determinístico, estático). Mas uma **partida** é uma **caminhada não-determinística** por esse espaço, por três fontes de variação:

1. **Achados reais são variáveis.** Rodar uma ferramenta contra o alvo produz saídas diferentes conforme o estado real do alvo — dois `web_crawl` podem revelar conjuntos diferentes de endpoints. Como as cartas-opção nascem dos achados (2.7), a mão diverge naturalmente.
2. **Arestas condicionais e ponderadas.** Cada aresta `enables` do grafo carrega uma **condição** (só dispara se o achado tiver certa propriedade) e um **peso** (probabilidade/prioridade de a opção emergir). Ex.: `INJ-SQLi` só "abre" `AUTH-Bypass` se a injeção tocar uma tabela de usuários; senão abre `POST-DataExfil`. Isso faz o mesmo nó gerar **ramos diferentes** conforme o contexto.

3. **Aleatoriedade controlada (opcional).** Para replayability e para os experimentos do artigo, o motor pode introduzir variação estocástica na ordem/surgimento de opções, governada por uma **seed**.

O resultado é uma **árvore emergente por partida**: começa em O Louco, e cada carta jogada "desenha" o próximo galho a partir do que foi realmente descoberto — nenhuma partida é igual. É o oposto de um playbook linear.

O ponto científico crucial (não-determinismo + reprodutibilidade): a partida é não-determinística para o *jogador*, mas **100% reproduzível** para a *ciência*. Como tudo (seed + cada achado + cada jogada) é gravado no EventLog (2.5), qualquer partida pode ser **re-executada idêntica** a partir do log. Você tem o melhor dos dois mundos: cenários emergentes e ricos para explorar, e reprodutibilidade total para o experimento. No artigo, isso vira uma seção forte: "geração procedural de cenários de pentest com replay determinístico".

Implementação: as condições/pesos vivem nas arestas do `attack_graph.yaml` (`enables: [{to: AUTH-Bypass, when: "finding.touches=='users'", weight: 0.8}, ...]`); `expand_options` (2.7) passa a avaliar `when` e ordenar/amostrar por `weight`; a seed entra no `game_rules.yaml` e é gravada no EventLog no início de cada partida.

2.17 Cada carta é um PACOTE (pasta autocontida) com acesso à caixa de ferramentas

Você quer que cada carta seja **uma pasta** no projeto, com seu `.md` e arquivos que você pode subir, e que cada carta-agente tenha **acesso às ferramentas** necessárias para executar seu teste e gerar o output que muda o campo. Este é o modelo de **card-package** — cada carta é um plugin autocontido.

Anatomia de uma pasta-carta

Tudo de uma carta vive em `cards/<arcano>/<id>/`. Exemplo real:

```
cards/
├── _base/                                # código compartilhado (ToolOperatorAgent, AgentResult)
├── mago/                                 # agrupado por ARCANO (2.5)
│   ├── A1-sql-injection/
│   │   ├── card.md                      # [] doc humana: o que faz, âncora, arcano, como testa, riscos
│   │   ├── card.yaml                   # schema 2.3 (id, arcana, badge, tool, preconditions, effects) – PRÉ-PREENCHIDO
│   │   ├── agent.py                    # subclasse ToolOperatorAgent (molde 2.13) – opera a ferramenta
│   │   ├── parser.py                   # traduz saída da ferramenta → findings estruturados
│   │   ├── payloads/                   # [] ARQUIVOS QUE VOCÊ SOBE: wordlists, payloads, configs da ferramenta
│   │   │   └── .gitkeep
│   │   └── tests/
│   │       └── test_agent.py            # teste contra alvo mock
│   ├── B1-reflected-xss/
│   └── ...
├── forca/
│   ├── C1-brute-force/
│   │   ├── card.md
│   │   ├── card.yaml
│   │   ├── agent.py
│   │   ├── payloads/                   # ex: você sobe aqui sua wordlist de senhas
│   │   └── tests/
│   └── ...
├── carro/E1-ssrf/ ...
├── torre/E2-file-upload/ ...
├── lua/G1-prompt-injection/ ...
└── justica/I1-validate-finding/ ...    # cartas-sistema também são pacotes
```

O que cada arquivo faz: **card.md** é a documentação (o texto que descreve a técnica, a âncora ATT&CK/OWASP, o arcano, o passo-a-passo do teste e os cuidados — é onde você lê/edita o "o quê e porquê"); **card.yaml** é a configuração que a engine consome; **agent.py** é o código que opera a ferramenta; **payloads/** é a pasta onde **você sobe seus arquivos** (wordlists, listas de payloads, templates de núclei, configs do sqlmap) — o agente lê dessa pasta automaticamente.

Auto-descoberta (plug-and-play)

O `registry.py` (orchestrator) faz um scan de `cards/**/card.yaml` no boot: toda pasta com um `card.yaml` válido **entra no jogo automaticamente**. Isso significa: para adicionar uma carta, basta soltar uma pasta nova em `cards/` (ou usar a Forja, 10.1, que gera a pasta inteira). Nada de registrar manualmente. Subir arquivos numa `payloads/` de uma carta existente também é detectado — a próxima jogada daquela carta já usa os novos arquivos.

Acesso às ferramentas — a caixa completa vive na VM de ataque

"Ter acesso a todas as ferramentas" é resolvido na camada de isolamento (Parte 8): a imagem `vm/Dockerfile.attack-box` **instala o toolkit COMPLETO** de pentest de uma vez (nmap, httpx, katana, ffuf, nuclei, sqlmap, dalfox, commix, tplmap, jwt_tool, hydra, subfinder, amass, sslscan, smuggler, trufflehog, graphw00f/InQL, garak, wafw00f, etc.). Assim **toda carta já encontra sua ferramenta instalada** — a carta apenas declara em `card.yaml` qual(is) usa, e o `ToolOperatorAgent` (2.12) a invoca. Três regras:

1. **Ferramenta do sistema:** está no PATH da VM (instalada pelo Dockerfile). O agente chama via subprocess. `config/tools.yaml` (2.12) mapeia nome→binário e diz se precisa de rede (`net:true` ⇒ exige VPN ligada).
2. **Ferramenta/insumo próprio da carta:** qualquer coisa específica (uma wordlist, um script custom, um payload set) vai em `payloads/` da própria pasta e é versionada/subida por você. O agente resolve o caminho relativo à sua pasta.
3. **Ferramenta ausente:** se o binário não estiver na VM, o agente devolve `status:"tool_missing"` (nunca falha silenciosa) — e o `card.md` lista o que instalar.

O ciclo output → campo (fechando o loop, agora por carta)

Quando você joga a carta, é o `agent.py` **daquela pasta** que roda:

```
cards/mago/A1-sql-injection/agent.py
→ seleciona a ferramenta (sqlmap) do tools.yaml, lê payloads/ da própria pasta
→ checa allowlist + VPN + timeout (Parte 8)
→ executa sqlmap contra o alvo concreto da carta-opção
→ parser.py traduz a saída → findings
→ devolve AgentResult.state_delta (ex: reveals:[vuln.sqli], unlocks:[Carro-... , Torre-...])
→ engine.apply_effects muda o Field → expand_options revela NOVAS cartas-opção no campo
→ cronista grava tudo no EventLog
```

Ou seja: a pasta é a unidade completa "documentação + config + agente + ferramentas/insumos + testes", e o output do agente é exatamente o que redesenha o campo e faz surgirem as próximas cartas. É isso que torna o baralho extensível — cada nova pasta é uma nova capacidade jogável.

Modelo de `card.md` (o que vai no `.md` de cada carta)

```
# A1 — SQL Injection · Arcano: O Mago (I)
**Âncora:** ATT&CK T1190 · OWASP A05:2025 (Injection)
**Aspecto:** sql  **Risco:** alto  **Custo (mana):** 3

## O que a carta faz
Testa injeção SQL nos parâmetros/forms descobertos no alvo autorizado.

## Ferramenta
sqlmap (instalada na attack-box). Insumos opcionais em `payloads/` (tamper scripts, wordlists).

## Pré-condições (quando entra na mão)
Requer `web.params_found > 0` ou `web.forms_found > 0` (derivado do attack graph).

## Efeitos no campo
Revela `vuln.sqli`; pode desbloquear DATA-Exfil e AUTH-Bypass.

## Cuidados / escopo
Só contra alvos da allowlist, VPN ligada, com limite de requisições. Achados passam por A Justiça.
```

2.18 Arsenal completo — o toolbelt das 22 cartas (47 ferramentas)

Cada arcano precisa **saber operar** as ferramentas reais do seu domínio. Abaixo o arsenal inteiro que você listou, distribuído pelos 22 arcanos. Todas ficam **instaladas na attack-box** (Parte 8/2.17) e registradas em `config/tools.yaml`; cada carta declara as suas e o `ToolOperatorAgent` (2.12) as invoca.

Escopo: esta lista amplia o jogo de "só web/links" para **web + rede + Active Directory + pós-exploração**. Isso enriquece a árvore (2.16): os arcanos finais (O Carro, O Diabo, A Torre) ganham profundidade de rede interna. Ferramentas GUI são operadas em modo **headless/CLI/API** pelo agente: Wireshark→tshark, ZAP→modo daemon/ zap-cli, Burp→REST API, Metasploit→msfconsole -x/RPC, Postman→newman.

Arcano	Fase	Ferramentas que a carta opera
O Louco (0)	recon (semente)	Nmap, RustScan
O Eremita (IX)	recon passivo/OSINT	Amass, Subfinder, theHarvester, Shodan, Maltego, DNSRecon, Masscan, enum4linux-ng
A Estrela (XVII)	recon TLS/cert	testssl.sh, SSLyze (+ <i>Certificate Transparency</i>)
A Imperatriz (III)	descoberta/crawl/params/API	httpx, ffuf, Gobuster, Feroxbuster, Dirsearch, Arjun, Postman/newman
A Sacerdotisa (II)	segredos/tráfego oculto	Wireshark/tshark, tcpdump, trufflehog (<i>JS</i> , <i>.git</i> , <i>backups</i>)
A Roda da Fortuna (X)	fingerprint + CVE conhecida	WhatWeb, Nuclei, Nikto, WPScan, Searchsploit
O Hierofante (V)	defesas/headers/config	Nikto (<i>compart.</i>), Nuclei (<i>templates de config</i>), testssl.sh (<i>compart.</i>)
O Mago (I)	injeções + XSS (core web)	Burp Suite, OWASP ZAP, SQLmap, Dalfox, XSSStrike
Os Enamorados (VI)	confiança/OAuth/JWT	jwt_tool, Burp (<i>fluxos OAuth/OIDC</i>)
A Força (VIII)	auth/senhas/hashtes	Hydra, Hashcat, John the Ripper
O Imperador (IV)	privilegio/admin/AD paths	BloodHound, PEASS-ng (LinPEAS/WinPEAS)
O Carro (VII)	movimentação lateral/pivoting/AD	Impacket, NetExec, ProxyChains, Chisel, Socat, Netcat
O Diabo (XV)	identidade/sessão/credenciais	Bettercap, Responder, Mimikatz
A Torre (XVI)	exploração crítica/RCE	Metasploit Framework, Netcat (<i>shells</i>), Searchsploit (<i>compart.</i>)
O Enforcado (XII)	evasão de WAF/filtro	wafw00f (+ <i>adaptação de payloads</i>)
A Lua (XVIII)	apps com LLM	garak (+ <i>sondas de prompt custom</i>)
Cartas-sistema (Justiça/Morte/Sol/Julgamento/Mundo)	engine	nenhuma ferramenta externa — operam sobre o estado/EventLog

Notas de integração: ferramentas compartilhadas (Nikto, testssl.sh, Searchsploit, Netcat, Burp) aparecem em mais de um arcano — no `tools.yaml` cada uma é definida **uma vez** e referenciada por vários `card.yaml`. Ferramentas de rede/AD (Impacket, Mimikatz, Responder, BloodHound) só fazem sentido depois que um acesso é obtido — suas cartas só entram na mão via arestas de pós-exploração do grafo (2.16), respeitando a linearidade das fases.

Reflexo no fluxo de gameplay

A distribuição acima **é** o arco da partida, e casa com as fases (a linearidade da árvore não-determinística):

1. **Abertura** — O Louco e O Eremita/A Estrela varrem e enumeram (Nmap, Subfinder, testssl.sh). O Sol consolida a superfície.
2. **Mapeamento** — A Imperatriz e A Sacerdotisa revelam endpoints, params e segredos (ffuf, Arjun, tshark), gerando muitas cartas-opção.
3. **Exploração** — A Roda (CVE), O Mago (injeções/XSS), A Força (auth), O Hierofante/Os Enamorados atacam o que foi revelado (Nuclei, SQLmap, Hydra, jwt_tool).
4. **Aprofundamento/pós** — quando um acesso cai, A Torre (Metasploit/RCE), O Carro (Impacket/pivoting), O Diabo (Mimikatz/Responder) e O Imperador (BloodHound/PEASS) abrem a rede interna.
5. **Validação e fecho** — A Justiça valida cada achado, O Julgamento formaliza, O Mundo fecha com o relatório do Cronista.

Cada ferramenta gera o output que o `parser.py` da carta converte em findings → muda o campo → revela as próximas cartas. É o mesmo loop de sempre, agora com o arsenal completo por trás.

Expansão do arsenal (ferramentas 51-100) e o novo domínio de nuvem

O modelo de 22 arquétipos comporta ferramentas ilimitadas — novas ferramentas só se **encaixam num arcano existente**, sem inflar o baralho. As adições vão assim:

Arcano	Novas ferramentas (51-100)
O Eremita	Assetfinder, SpiderFoot, Naabu, dnsx, ZMap, Netdiscover, arp-scan
A Imperatriz	GAU, Waybackurls, Katana, Hakrawler, ParamSpider, Wfuzz, Kiterunner, GraphQLmap, InQL, Aquatone, EyeWitness <i>(screenshots → também alimentam O Sol)</i>
A Sacerdotisa	LinkFinder, SecretFinder, Gitleaks, mitmproxy <i>(inspeção)</i>
O Mago	Caído, NoSQLMap, CRLFuzz <i>(injeções/proxy)</i>
A Torre	Commix, Tplmap, SSTImap, ysoserial, marshalsec, Evil-WinRM <i>(RCE/shell)</i>
O Hierofante	Corsy, ScoutSuite <i>(config/CORS/nuvem)</i>
A Temperança	Smuggler <i>(request smuggling)</i>
A Roda da Fortuna	Nessus, OpenVAS/Greenbone, Qualys VMDR, Invicti, Acunetix, Trivy <i>(scanners de CVE — web/infra/containers)</i>
A Força	Kerbrute <i>(Kerberos)</i>
O Carro	Scapy, Hping3, PowerView, kube-hunter <i>(rede/AD/K8s lateral)</i>
O Diabo	Rubeus <i>(abuso Kerberos)</i>
O Imperador	SharpHound, Certipy, Prowler <i>(coleta AD / AD CS / IAM de nuvem)</i>

Novo domínio — Nuvem / Containers / Kubernetes. Prowler, ScoutSuite, Trivy e kube-hunter abrem um ramo que **não existia**. Ele se conecta ao grafo pela ponte que já tínhamos: SSRF → POST-CloudMetadata (2.10/2.11) — ao capturar credenciais de metadados de nuvem, a partida entra na fase de nuvem, onde O Imperador audita IAM (Prowler), O Hierofante checa config (ScoutSuite), A Roda varre imagens (Trivy) e O Carro pivota no cluster (kube-hunter). Adicione ao attack graph os nós CLOUD-IAMAudit, CLOUD-ConfigAudit, CLOUD-ImageCVE, K8S-ClusterAttack, alcançáveis a partir de POST-CloudMetadata.

[!] **Ferramentas comerciais/licenciadas** (Nessus, Qualys VMDR, Invicti, Acunetix, Burp Suite Pro, Shodan) **não são empacotadas** na attack-box — exigem licença/API key. No tools.yaml marque-as licensed: true; a carta que as usa cai em status:"tool_unavailable" até o usuário fornecer a chave (guardada no keyring, Parte 10.6). As alternativas livres cobrem o mesmo aspecto: OpenVAS↔Nessus, ZAP/Caído↔Burp, Nuclei↔Acunetix/Invicti — então o jogo é 100% funcional só com ferramentas open-source.

Com isso o arsenal registrado chega a ~100 ferramentas cobrindo recon, web, API, rede, AD, pós-exploração e nuvem — todas sob os mesmos 22 arcanos.

Rodada 3 do arsenal — ofensivas novas + um domínio NOVO (reputação/phishing)

A maioria da sua última lista (Burp, ZAP, Nuclei, httpx, ffuf, SQLmap, Dalfox, testssl, etc.) **já está coberta**. As **ofensivas realmente novas** encaixam assim:

Arcano	Ofensivas novas
A Roda da Fortuna	Wapiti, JoomScan, Droopescan, Wappalyzer, Webanalyze, Retire.js
O Hierofante	CORScanner, Mozilla Observatory, SecurityHeaders.com
Os Enamorados	OpenRedireX, Oralyzer <i>(open redirect)</i>
O Carro	SSRFmap, Gopherus <i>(SSRF/exploração)</i>
O Imperador	Autorize, AuthMatrix <i>(controle de acesso)</i>
A Temperança	HTTP Request Smuggler <i>(ext. Burp)</i>
A Estrela	SSLscan, Qualys SSL Labs

[!] **Domínio NOVO — Reputação, phishing e malware (61-75) é outra coisa.** Preciso ser direto: VirusTotal, urlscan.io, Google Safe Browsing, PhishTank, OpenPhish, ANY.RUN, Hybrid Analysis, Joe Sandbox, URLVoid, etc. **não são pentest de um alvo autorizado** — são **threat intelligence / triagem passiva de um link desconhecido** (consultam serviços de reputação de terceiros, sem tocar o alvo). É a diferença entre "atacar uma aplicação que eu tenho permissão para testar" e "avaliar se um link recebido é malicioso".

Isso conecta com a ideia original ("o inimigo é o link"), mas em outro **modo de jogo**:

- **Modo Pentest** (o que construímos): ataca um alvo na allowlist, com VPN, exploração ativa.

- **Modo Triagem de Link** (novo, opcional): dado um link suspeito, faz uma leitura **100% passiva** de reputação/comportamento — sem allowlist, sem VPN, sem exploração (só consulta APIs externas e sandboxes). Seguro por natureza.

Encaixo esse modo como uma **carta de abertura sob O Eremita** (recon passivo/OSINT) chamada `TRIAGE-LinkReputation`, que agrega as fontes de reputação num veredito (limpo/suspeito/malicioso + evidências) — útil como "primeiro olhar" antes de decidir atacar, e como um sub-jogo autônomo de análise de phishing. A maioria dessas fontes é **API comercial/licenciada** (VirusTotal, urlscan, Joe Sandbox, ANY.RUN, IPQualityScore): marque `licensed:true`; sem chave, a carta usa só as fontes livres (Google Safe Browsing, PhishTank, OpenPhish, URLVoid).

Decisão de design que deixo em aberto: manter a Triagem de Link como **um segundo modo** (recomendado — mantém o Modo Pentest limpo e ainda aproveita as ferramentas) ou descartá-la se você quer o projeto 100% focado em pentest ofensivo. Me diga qual prefere.

Rodada 4 — ferramentas de infraestrutura que ELEVAM a proposta (não só mais scanners)

Estas são diferentes das anteriores: várias não são "mais uma carta", são **capacidades de infraestrutura** que várias cartas compartilham e que aumentam o *rigor* dos resultados. Minha avaliação de valor:

Altíssimo valor (adote todas) — P0:

- **Interactsh** `☐` (o mais importante da lista). Servidor de callback OOB (DNS/HTTP/SMTP) que **confirma vulnerabilidades cegas** — blind SQLi, SSRF, XSE, blind XSS, command injection. Sem ele, essas classes são chute; com ele, viram achado *comprovado*. Não é uma carta: é um **serviço da attack-box** que muitas cartas usam e que alimenta diretamente **A Justiça** (a validação de vuln cega vira "recebi o callback X às HH:MM"). Isso resolve, na prática, o problema de falso-positivo do paper "Hackers or Hallucinators?". Implementação: um listener roda na VM; o payload de cada carta cega injeta um domínio único; callbacks entram no EventLog e A Justiça correlaciona. Usado por O Mago, O Carro (SSRF), A Torre (cmd injection).
- **Playwright + Chromium** `☐`. Motor de browser real (executa JS, interage como usuário). É o que torna possíveis **DOM XSS, CSRF, OAuth/MFA, sessão, WebSocket e fluxos de negócio** — coisas impossíveis só com HTTP cru. Capacidade compartilhada: cartas que precisam de browser (O Mago/DOM, Os Enamorados/OAuth-MFA, O Diabo/sessão-WS, lógica de negócio) operam via Playwright em vez de subprocess.
- **Schemathesis**. Fuzzing por propriedades a partir de OpenAPI/GraphQL — cobre a família API (mass assignment, broken auth, validação, business flow) de forma sistemática. Vai para **A Imperatriz** (descoberta/fuzzing de API).
- **SecLists**. O corpus padrão de wordlists/payloads. Não é ferramenta, é **o conteúdo default das payloads/** de muitas cartas (ffuf, Hydra, Gobuster, Arjun, fuzzing). Baixo custo, valor enorme — vira recurso compartilhado em `cards/_shared/seclists/`.
- **curl + jq + yq**. A cola das cartas custom (requisição + parsing confiável de JSON/YAML). Utilitários-base da VM; todo agente pode usar.

Forte valor (adote conforme a fase) — P1:

- **websocat** → **O Diabo** (WebSocket hijacking/autorização via CLI).
- **ZAP Automation Framework** → baseline automatizado + **validação cruzada**. Ótimo par para **A Justiça**: um segundo motor independente confirmando o achado reduz falso-positivo (e vira métrica de concordância no artigo).
- **Autorize / AuthMatrix (ou runner próprio)** `☐` → **O Imperador**. É a forma correta de testar IDOR/BOLA/BFLA/escalada: comparar respostas entre usuários/permisões diferentes (teste diferencial). Bugs de controle de acesso praticamente exigem isso.
- **subzy / nuclei takeover templates** → **O Eremita** (alimenta o nó `SC-SubdomainTakeover` já existente).
- **Semgrep** `☐` → **A Sacerdotisa** + **A Justiça**. Traz um modo **grey-box**: quando o lab fornece o código-fonte, análise estática acha code injection/secrets/deserialização e **valida achados dinâmicos contra o código**. Diferencial científico (comparar caixa-preta vs. grey-box).

Valor situacional — P2 (só se o escopo incluir):

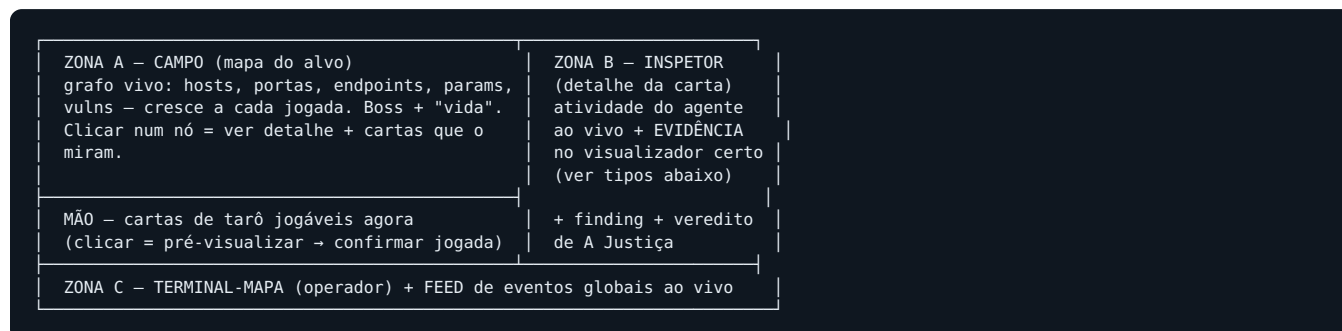
- **Kubescape + kube-bench** → domínio de nuvem/K8s (O Hierofante/O Carro), complementam o kube-hunter com benchmarks.
- **Checkov** → **O Hierofante** (misconfig em Terraform/K8s/IaC), se o lab fornecer IaC.
- **Dradis CE / Faraday** → **recomendo NÃO adotar**. Você está certo: seu **Cronista + EventLog + artefatos tipados (2.19)** já fazem consolidação de evidência e relatório, de forma mais integrada e reproduzível. No máximo, deixe um *exporter* opcional para o formato deles se um dia precisar de fluxo corporativo.

Resumo da recomendação: **Interactsh**, **Playwright**, **SecLists**, **curl/jq/yq** entram já (mudam o que é possível e o rigor). **Authorize-style**, **Semgrep** e **ZAP-AF** entram cedo (controle de acesso, grey-box, validação cruzada). **K8s/IaC** entram com o domínio de nuvem. **Dradis/Faraday** ficam de fora — o Cronista já resolve.

2.19 Interação e apresentação — o que o usuário VÊ e FAZ (a peça que faltava)

Pergunta certa: se cada agente faz uma coisa diferente (um escaneia portas, outro extrai dados com SQLi, outro tira screenshots, outro monta um grafo de Active Directory), **um terminal não basta**. A solução é: a UI **não renderiza "um terminal"** — ela renderiza a **atividade estruturada de cada agente** e mostra a **evidência com o visualizador certo para cada tipo**. O terminal é apenas *um* dos formatos, não o todo.

Layout da tela — três zonas



- **Zona A (Campo):** a representação visual do **estado real do sistema de teste** — um grafo/mapa que cresce: cada host, porta, endpoint, parâmetro e vulnerabilidade descoberto vira um nó. É aqui que "o que está acontecendo no alvo" fica visível de forma agregada. O boss perde "vida" conforme a superfície é mapeada/comprometida.
- **Zona B (Inspetor):** quando você seleciona uma carta jogada (ou um achado), abre o detalhe **daquele agente**: o que ele fez, passo a passo, e a evidência renderizada.
- **Zona C (Terminal + feed):** a visão crua de operador e um feed cronológico de todos os eventos (útil para acompanhar o ritmo e para quem gosta de texto).

Como cada agente reporta o que faz (contrato uniforme)

Todo agente — independentemente de rodar `nmap`, chamar a API do Burp, ou raciocinar com LLM — emite dois fluxos padronizados (adição ao schema 2.3):

1. **Activity stream** — uma sequência de *passos* estruturados durante a execução, transmitidos ao vivo por WebSocket: `{ts, phase, level, message, artifact_ref?}`. Ex.: `[recon] iniciando nmap -sV`, `[recon] porta 3000/tcp aberta`, `[analysis] LLM classificou resposta como injetável`. Isso dá uma **timeline legível** de qualquer agente, mesmo os que não são "terminais".
2. **Artifacts tipados** — as saídas reais, cada uma com um `type` que diz à UI **como renderizar**. É isto que resolve a heterogeneidade.

Tipos de artefato → visualizador (o coração da resposta)

Artefato	Produzido por (ex.)	Como a UI mostra
log	qualquer ferramenta CLI	bloco de console (é o "terminal" daquele passo)
table	Nmap, ffuf, enum de usuários	grade ordenável (porta/serviço, endpoint/status)
http_exchange	SQLmap, Burp, XSS, SSRF	painel requisição/resposta lado a lado (estilo Burp), com o payload destacado
screenshot	EyeWitness, Aquatone	visualizador de imagem (galeria de telas dos serviços)
graph	BloodHound/SharpHound	grafo de nós navegável (caminhos até admin)
json	APIs, GraphQL introspection	árvore JSON colapsável
file	dump de dados, código do <code>.git</code> , secrets	preview + download seguro
diff	O Enforcado (WAF bypass), before/after	comparação lado a lado (resposta bloqueada vs. contornada)
metric	O Sol, cobertura, custo	medidor/número (ex: "cobertura 42%", "req/achado")
finding	qualquer exploração	cartão de achado: tipo, severidade, evidência, âncora ATT&CK/OWASP, status da A Justiça

Assim, jogar **A Estrela** (TLS) mostra uma `table` de cifras; **O Diabo** (BloodHound) mostra um `graph`; **A Imperatriz** (EyeWitness) mostra `screenshot`s; **O Mago** (SQLi) mostra `http_exchange` + `file` (dados extraídos). Cada agente escolhe o(s) tipo(s) que fazem sentido — a UI já sabe desenhar todos.

A interação do usuário com o campo (passo a passo)

1. **Explorar o campo:** clicar num nó descoberto (ex: um endpoint) destaca **quais cartas o miram** e mostra o que já se sabe dele.
2. **Pré-visualizar uma carta:** clicar numa carta da mão abre um preview — ferramenta que vai usar, alvo concreto, custo de mana, efeito esperado, risco. Nada roda ainda.
3. **Jogar:** confirmar dispara o agente. A Zona B mostra a *activity stream* ao vivo e os artefatos vão aparecendo; a Zona A anima o surgimento dos novos nós; a Zona C loga tudo.
4. **Inspecionar evidência:** clicar num achado abre a evidência tipada (o request/response, a screenshot, o grafo) e o veredito de A Justiça (confirmado/falso-positivo/severidade).
5. **Alternativa por teclado:** tudo isso também via terminal (`play`, `inspect <finding>`, `map`), para quem prefere fluxo de operador.

Por que isso é melhor que "só um terminal"

O terminal continua lá (Zona C, e como `artifact.type=log`), mas ele é o **piso**, não o teto. O usuário entende o que está acontecendo em três níveis: **macro** (o mapa do alvo crescendo na Zona A), **médio** (a timeline de passos de cada agente na Zona B) e **micro** (a evidência crua — request/response, screenshot, grafo, arquivo). É isso que faz agentes que fazem coisas radicalmente diferentes caberem numa mesma interface coerente: contrato de atividade uniforme + visualizador por tipo de evidência.

Implementação: adicione ao `AgentResult` (2.3) os campos `activity: [ActivityEvent]` e `artifacts: [{type, ref, meta}]`. O `api/server.py` (Sprint 5) transmite `ActivityEvent`s pelo `WebSocket` em tempo real; o frontend tem um componente `ArtifactViewer` que despacha por `type`. O Cronista (2.5) reaproveita os mesmos artefatos tipados no relatório final.

2.5 O Cronista (documentador) — o relatório que se escreve durante a partida

Não é opcional — é requisito, e é uma das peças mais importantes para o valor prático e científico. O ponto central que você pediu: **o relatório vai sendo construído ao longo de todo o processo** (não só no fim), fica **visível ao vivo**, e ao terminar é **baixável, organizado e formatado, explicando todo o caminho de como se chegou a cada falha**.

Construção progressiva (ao vivo)

O Cronista **não espera o fim**. Ele escuta o `EventLog` em tempo real e, a cada jogada, **acrescenta uma entrada ao relatório**: qual carta foi jogada, sobre qual alvo, o que o agente fez (a *activity stream*, 2.19), qual evidência produziu e o que mudou no campo. Assim existe sempre um "relatório vivo" — uma aba/painel na UI (Zona B) que você pode abrir a qualquer momento e ver o documento crescendo. É o mesmo material que, no fim, vira o arquivo final.

O "caminho até a falha" (a reconstrução da trilha de ataque)

Cada achado confirmado carrega sua **provreniência completa**: a sequência de cartas/nós que levou até ele, do início (O Louco) até a exploração. Como tudo vem do grafo (2.8) e do EventLog, o Cronista **reconstrói a trilha** e a apresenta como uma narrativa passo a passo — "por que esta falha foi alcançável":

```
Caminho até "SQL Injection em /rest/products?q" (severidade alta):
0 Louco → recon passivo revelou o host
A Imperatriz (Web Crawl) → descobriu o endpoint /rest/products e o parâmetro q
A Roda da Fortuna → fingerprint indicou stack sem WAF
0 Mago (SQLi) → confirmou injeção error-based no parâmetro q
A Justiça → validou (callback Interactsh recebido) · sem detecção pelo alvo · CVSS 8.6
0 Julgamento → achado formalizado com evidência e recomendação
```

Cada passo linka para a evidência tipada (o request/response, o log da ferramenta) e para a âncora ATT&CK/OWASP. Isso é exatamente o "como chegou até a falha" — reproduzível e auditável, porque a trilha é derivada do log, não escrita à mão.

Estrutura do relatório final

Resumo executivo · Escopo e ética (allowlist, seed, ambiente) · **Trilha de ataque por achado** (as narrativas acima) · Achados por severidade com evidência e âncora · Superfície coberta vs. total · Cartas jogadas vs. disponíveis · Custo (mana/requisições/tempo) · Recomendações de correção por achado · Apêndice reproduzível (seed + hash do EventLog para replay idêntico).

Baixável e formatado

Ao acionar **O Mundo** (ou a qualquer momento), o relatório é exportado em **Markdown → PDF** (pipeline da skill de PDF) e também **HTML** — botão "Baixar relatório" na UI. Layout limpo, com sumário, seções e as evidências embutidas (tabelas, request/response, screenshots).

Regra de integridade

Pode ser **LLM-assistido para a prosa** (deixar a narrativa fluida), mas os **fatos vêm do EventLog**, nunca do modelo — números, evidências e a trilha são derivados do log. Isso evita alucinação e mantém o relatório defensável no artigo.

2.6 O Terminal como "mapa" da partida (Linux embutido)

Além do board de cartas, a interface tem um **terminal Linux real embutido**, que funciona como o **mapa** que o usuário lê para se orientar: ele mostra o "terreno" (estado do alvo) e recebe comandos de navegação/inspeção. Duas funções:

1. **Mapa vivo (saída)**: o terminal reflete cada jogada em tempo real — como um log de exploração no estilo de um shell de operador. Ex.: ao jogar `RECON-Portas`, ele imprime a saída do scan; ao revelar um endpoint, escreve `[+] novo caminho descoberto: /rest/products`. É a "mini-mapa textual" do campo de batalha.
2. **Navegação (entrada)**: o usuário digita comandos para consultar/mover-se pelo mapa sem precisar do mouse. Ex.: `map` (mostra o campo atual), `hand` (lista cartas jogáveis), `play web_sql_i`, `inspect boss`, `report`. Esses comandos batem no mesmo orquestrador que o board — board e terminal são duas vistas do mesmo estado.

Como funciona (arquitetura do terminal)

- **Frontend**: `xterm.js` renderiza o emulador de terminal dentro do React (`ui/src/terminal.tsx`).
- **Backend**: um **PTY Linux real** por sessão, aberto pelo servidor com o módulo `pty` do Python (ou `node-pty`). O shell roda dentro de um **container Linux** (o mesmo `docker-compose` dos labs), garantindo o "terminal de configuração Linux" isolado e reproduzível — **nunca** o shell da máquina do usuário.
- **Transporte**: um WebSocket dedicado (`/ws/terminal`) faz o duplex: teclas do `xterm.js` → PTY; bytes do PTY → `xterm.js`. Reaproveita a stack de WebSocket do Sprint 5.
- **Duas camadas de comando**:
 - **Comandos do jogo** (`map`, `hand`, `play ...`, `report`) são interceptados por um pequeno REPL/shell wrapper que chama o orquestrador e imprime a resposta formatada. É por aqui que o terminal vira "mapa".
 - **Comandos de shell Linux** (`ls`, `cat`, `nmap`, `curl`, ferramentas de pentest) rodam no PTY do container — **restritos à allowlist**: qualquer comando que tente alcançar um host fora de `config/allowlist.yaml` é bloqueado pelo wrapper antes de chegar ao PTY.
- **Segurança**: o container roda sem privilégios, com rede limitada aos alvos de lab; a mesma allowlist da Parte 0 vale aqui. O terminal é sandbox, não acesso ao host.

Assim o "mapa" é literal: um shell Linux onde o estado do pentest é o terreno, os comandos do jogo são a bússola, e cada carta jogada desenha uma nova região no mapa.

PARTE 3 — Estrutura de pastas do repositório

```

pentest-arena/
├── README.md
├── ETHICS.md
├── pyproject.toml
├── docs/
│   ├── literature/
│   │   ├── 00-standards/
│   │   │   └── # ATT&CK, ATLAS, PTES, OWASP (baixados)
│   │   ├── 01-papers-agentes/
│   │   │   └── # PentestGPT, VulnBot, surveys (arXiv)
│   │   ├── 02-papers-fundacao/
│   │   │   └── # ReAct, Toolformer, Reflexion
│   │   ├── 03-livros/
│   │   │   └── # só BIBLIOGRAFIA.md com citações (livros comerciais não entram)
│   │   └── 04-ferramentas/
│   │       └── # docs oficiais das ferramentas usadas pelos agentes
│   ├── config/
│   │   ├── allowlist.yaml
│   │   │   └── # alvos permitidos
│   │   ├── game_rules.yaml
│   │   │   └── # objetivo, vitória/derrota, economia de mana (Parte 10.5)
│   │   ├── settings.py
│   │   │   └── # pydantic-settings: config central + segredos (Parte 10.6)
│   │   ├── knowledge/
│   │   │   ├── attack_graph.yaml
│   │   │   │   └── # grafo de ataque real (ATT&CK/PTES/OWASP) – fonte das cartas (2.8)
│   │   │   └── arcana.yaml
│   │   │       └── # mapa Arcano Maior → função/nós (Parte 5.2)
│   │   ├── vpn/
│   │   │   ├── .gitkeep
│   │   │   └── lab.ovpn.example
│   │   │       └── # [!] GITIGNORED – perfis/segredos de VPN nunca versionados
│   │   └── tools.yaml
│   │       └── # exemplo de config (sem segredos reais)
│   ├── cards/
│   │   ├── _base/
│   │   │   └── base.py
│   │   │       └── # registro de ferramentas + mapa nó-ferramenta (2.12)
│   │   ├── shared/
│   │   │   └── # □ CADA CARTA É UMA PASTA AUTOCONTIDA (Parte 2.17)
│   │   │       └── # código compartilhado: CardAgent + ToolOperatorAgent (2.12/2.13)
│   │   ├── seclists/
│   │   │   └── # recursos compartilhados (2.18-Rodada 4)
│   │   ├── interactsh/
│   │   │   └── # SecLists (wordlists/payloads default)
│   │   ├── browser/
│   │   │   └── # cliente do listener OOB (confirma vulns cegas)
│   │   ├── louco/H1-recon-seed/
│   │   │   └── # runner Playwright/Chromium (DOM/OAuth/MFA/WS/lógica)
│   │   ├── eremita/H2-osint-dns/
│   │   │   └── # cada pasta: card.md + card.yaml + agent.py + parser.py + payloads/ + tests/
│   │   ├── imperatriz/H4-web-crawl/
│   │   │   └── #
│   │   ├── mago/
│   │   │   ├── A1-sql-injection/
│   │   │   │   └── # card.md, card.yaml, agent.py, parser.py, payloads/, tests/
│   │   │   ├── B1-reflected-xss/
│   │   │   │   └── #
│   │   │   └── ...
│   │   ├── forca/C1-brute-force/
│   │   │   └── # payloads/ = onde você sobe wordlists
│   │   ├── imperador/D4-idor-bola/
│   │   ├── carro/E1-ssrf/
│   │   ├── torre/E2-file-upload/
│   │   ├── lua/G1-prompt-injection/
│   │   └── justica/I1-validate-finding/
│   │       └── # cartas-sistema também são pacotes
│   ├── engine/
│   │   ├── field.py
│   │   │   └── # estado do campo + event sourcing
│   │   ├── game_engine.py
│   │   │   └── # turnos, mão jogável, condições, when/weight (2.16)
│   │   ├── knowledge.py
│   │   │   └── # carrega attack_graph.yaml; consultas children_of/reachable_from (2.8)
│   │   ├── events.py
│   │   │   └── # tipos de evento + JSONL log
│   ├── orchestrator/
│   │   ├── orchestrator.py
│   │   │   └── # roteia jogada-agente, aplica allowlist/VPN/orçamento
│   │   └── registry.py
│   │       └── # AUTO-DESCOBERTA: scan de cards/**/card.yaml no boot (2.17)
│   ├── chronicler/
│   │   ├── chronicler.py
│   │   │   └── # consome log → relatório
│   │   └── templates/report.md.j2
│   │       └── # cria a PASTA-CARTA inteira em cards/ (Parte 10.1 + 2.17)
│   ├── forge/
│   │   ├── forge.py
│   │   │   └── # card.md.j2 + card.yaml.j2 + agent.py.j2 + test_agent.py.j2
│   │   └── templates/
│   │       └── # integração Claude p/ cartas llm_assisted (Parte 10.3)
│   ├── llm/
│   │   ├── client.py
│   │   └── prompts/
│   ├── storage/
│   │   ├── db.py
│   │   │   └── # SQLite: partidas, cartas forjadas, métricas (Parte 10.2)
│   ├── packaging/
│   │   ├── pyinstaller.spec
│   │   └── build.sh
│   │       └── # scripts de build do binário (Parte 11)
│   ├── api/
│   │   ├── server.py
│   │   │   └── # FastAPI + WebSocket p/ o frontend
│   │   ├── terminal.py
│   │   │   └── # PTY Linux por sessão + WS /ws/terminal + wrapper de comandos
│   │   ├── vpn.py
│   │   │   └── # gerencia o daemon VPN + expõe status (ver Parte 8)
│   │   ├── preflight.py
│   │   │   └── # status consolidado VM/Docker/VPN/tools (Central + Segurança, P13)
│   │   ├── labs_control.py
│   │   │   └── # Docker control API: start/stop/snapshot/health (Laboratório, P13)
│   │   └── audit.py
│   │       └── # feed de ações bloqueadas para o Centro de Segurança (P13)
│   ├── ui/
│   │   └── src/
│   │       ├── Board.tsx
│   │       │   └── # campo de batalha
│   │       ├── Hand.tsx
│   │       │   └── # cartas jogáveis
│   │       ├── Card.tsx
│   │       │   └── # carta = sprite do ARCANO (arcana_id) + badge da técnica (5.3)
│   │       ├── Inspector.tsx
│   │       │   └── # ZONA B: activity stream do agente + evidência (2.19)
│   │       ├── ArtifactViewer.tsx
│   │       │   └── # despacha por type: log/table/http_exchange/screenshot/graph/... (2.19)
│   │       ├── EventFeed.tsx
│   │       │   └── # feed cronológico global de eventos (2.19)
│   │       ├── Terminal.tsx
│   │       │   └── # xterm.js = o "mapa" (ver 2.6)
│   │       ├── VpnToggle.tsx
│   │       │   └── # botão verde/vermelho (ver Parte 8)
│   │       ├── screens/
│   │       │   └── # as 12 telas (Parte 13) via react-router
│   │       │       ├── Command.tsx
│   │       │       │   └── # /command – Central de Comando
│   │       │       ├── Atlas.tsx
│   │       │       │   └── # /atlas – Atlas do Ataque (grafo completo)
│   │       │       ├── Grimoire.tsx
│   │       │       │   └── # /grimoire – Grimório das cartas
│   │       │       ├── Labs.tsx
│   │       │       │   └── # /labs – Laboratório de Alvos
│   │       │       ├── Evidence.tsx
│   │       │       │   └── # /evidence – Mesa de Evidências
│   │       │       └── Replay.tsx
│   │       │           └── # /replay – Linha do Tempo / Replay

```

Report.tsx	# /report – Sala do Cronista
Metrics.tsx	# /metrics – Observatório
Compare.tsx	# /compare – Arena Comparativa
Justice.tsx	# /justice – Sala da Justiça
Oracle.tsx	# /oracle – Oráculo
Security.tsx	# /security – Centro de Segurança (overlay sempre acessível)
ws.ts	
public/sprites/	
arcana/	# ← 22 sprites de tarô do Codex (louco.png ... mundo.png)
assets/	
sprites/	
arcana/	# PNG/SVG mestre dos 22 Arcanos Maiores
tests/	
test_field.py	
test_engine_hand.py	
test_agents_mock.py	# agentes contra alvo mock (sem rede real)
test_vpn_gate.py	# prova o kill-switch (Parte 9.4)
labs/	
docker-compose.yml	# sobe Juice Shop/DVWA local + rede isolada
vm/	
Dockerfile.attack-box	# VM de ataque: INSTALA TODO O ARSENAL (2.18) – base Kali/ParrotOS
	# nmap, rustscan, masscan, amass, subfinder, theharvester, shodan,
	# dnsrecon, whatweb, httpx, nuclei, ffuf, gobuster, feroxbuster,
	# dirsearch, nikto, sqlmap, wpscan, dalfox, xssstrike, testssl, sslyze,
	# arjun, jwt_tool, burp, zap, metasploit, searchsploit, netcat, socat,
	# tshark, tcpdump, bettercap, proxychains, chisel, hydra, hashcat, john,
	# impacket, netexec, bloodhound, responder, mimikatz, enum4linux-ng,
	# peass-ng, wafw00f, graphw00f, garak (base Kali cobre a maioria)
README.md	# como a VM se conecta ao host (ver Parte 8)
.gitignore	# DEVE incluir config/vpn/*.ovpn, *.key, *.crt

PARTE 4 — Divisão de trabalho: fable (VSCode) vs. Codex

- **fable (eu), no VSCode, faço:** toda a lógica e a **integração** — engine, orquestrador, agentes, cronista, API, componentes React, testes e depuração. Sou o construtor principal e o responsável por ligar as peças respeitando os contratos da Parte 0.1.
- **Codex faz apenas:** geração dos **22 sprites de tarô** (um por Arcano Maior — Parte 5) e, opcionalmente, boilerplate repetitivo (ex: scaffolding das 22 pastas-carta). Você gera as imagens com os prompts da Parte 5.4, salva em `assets/sprites/arcana/` e `ui/public/sprites/arcana/`, e eu apenas as referencio no `Card.tsx` pelo `arcana_id`. São 22 artes cobrindo os ~90 nós (não 90 imagens).

Motivo: separação limpa. Eu não gasto esforço "desenhando" — consumo assets prontos por caminho de arquivo; o Codex não decide arquitetura nem integra módulos.

PARTE 5 — Sprites com Codex (assets visuais)

5.1 Direção de arte: TARÔ (Arcanos Maiores)

A temática visual é **tarô** — especificamente os 22 **Arcanos Maiores**. Cada arcano é um **arquétipo** que representa um papel/ momento do pentest. Esta escolha resolve um problema real de escala: são **22 sprites** (um por arcano), não ~90 (um por nó). Cada um dos ~90 nós do grafo (Partes 2.9–2.11) é **etiquetado com um arcano** e reutiliza o sprite daquele arcano, com um pequeno **selo/rótulo** indicando a técnica concreta. Você desenha 22 cartas belíssimas em vez de 90 medianas, e ganha coerência visual imediata.

Estilo: inspirado na tradição Rider-Waite-Smith (a mais icônica), mas **original** — não copie as ilustrações protegidas de nenhum baralho comercial. Linha de contorno firme, simbolismo denso, paleta rica com dourados/ocres e um azul-noite profundo, moldura ornamentada com o numeral romano no topo e o nome do arcano na base. Releitura cyber-esotérica: os símbolos clássicos do tarô se fundem com elementos de tecnologia/rede (o cajado do Mago é uma antena; a lâmpada do Eremita é um terminal brilhando; a Torre é um servidor sendo partido por um raio).

5.2 Mapa dos 22 Arcanos Maiores → função no jogo e nós do grafo

Ajustei algumas funções que você propôs para encaixarem melhor na engine (marcado com ✳). Coluna **Tipo:** `técnica` = agente que executa ferramenta; `sistema` = carta de mecânica/meta (mexe no fluxo, não ataca).

#	Arcano	Arquétipo	Tipo	Função no jogo	Nós do grafo cobertos
0	O Louco	louco	sistema	Início da investigação; primeiro recon passivo, abre os caminhos iniciais do grafo.	RECON-PassiveDNS , nó-semente inicial
I	O Mago	mago	sistema	❖ O arquétipo do ToolOperatorAgent (2.12) — a identidade visual de toda carta que opera uma ferramenta. Não é um nó único; é o "skin" das cartas-técnica.	(transversal a todas as cartas técnica)
II	A Sacerdotisa	sacerdotisa	técnica	Procura o oculto: JS, metadados, arquivos expostos, comentários, chaves.	RECON-JSAnalysis , CRYPTO-ExposedSecrets , MISC-ExposedGit , MISC-BackupFiles
III	A Imperatriz	imperatriz	técnica	Web Crawl: revela páginas, endpoints, formulários, parâmetros e APIs (gera muitas cartas-opção).	RECON-WebCrawl , RECON-ParamDiscovery , RECON-HiddenAPI , API-SchemaDiscovery
IV	O Imperador	imperador	técnica	Controle administrativo: acesso a funções privilegiadas/áreas admin.	MISC-ExposedAdmin , BAC-MissingFunctionAuth , API-BFLA , POST-AdminAccess
V	O Hierofante	hierofante	técnica	Regras e padrões defensivos: headers, CSP, HSTS, CORS, X-Frame-Options.	MISC-SecurityHeaders , MISC-CORSCheck , CLICKJACK
VI	Os Enamorados	enamorados	técnica	Relações de confiança: OAuth, SSO, redirecionamentos, consentimento, vinculação de contas.	OAuth-Attack , EXC-OpenRedirect , BAC-CrossOrigin , API-UnsafeConsumption
VII	O Carro	carro	técnica	Movimentação lateral: usa sessão/credencial obtida para investigar outros serviços.	POST-LateralMovement , POST-Pivot , RECON-InternalScan , POST-CloudMetadata
VIII	A Força	força	técnica	Resistência da autenticação: brute-force controlado, spraying, credenciais padrão.	AUTH-CredBruteforce , AUTH-CredStuffing , AUTH-DefaultCreds , MFA-Bypass
IX	O Eremita	eremita	técnica	Recon silencioso: OSINT, DNS, subdomínios, dados públicos, sem exploração.	RECON-OSINT-Dorking , RECON-PassiveDNS , RECON-SubdomainEnum
X	A Roda da Fortuna	roda	técnica	Oportunidades por tecnologia: correlaciona versões/componentes com CVEs conhecidas.	RECON-TechFingerprint , SC-VulnComponent , SC-KnownCVE-Exploit
XI	A Justiça	justica	sistema	❖ Agente Validador (novo — ver 5.5) : confirma evidência, filtra falso-positivo, checka se o alvo detectou/logou, calcula severidade.	VALIDATE-Finding (nó novo), LOG-DetectionCheck
XII	O Enforcado	enforcado	técnica	Muda a abordagem quando bloqueado: adapta payloads, avalia WAF/filtro.	EVASION-WAFBypass , RECON-WAFDetect
XIII	A Morte	morte	sistema	❖ Transição de fase (mecânica da engine) : encerra um ramo e promove vuln confirmada → exploração/pós-exploração.	(aciona apply_effects de mudança de fase)
XIV	A Temperança	temperanca	técnica	Diferenças entre componentes que deveriam concordar: HTTP Request Smuggling, cache.	HTTP-RequestSmuggling , CACHE-Poisoning
XV	O Diabo	diabo	técnica	Abuso de identidade/confiança: roubo/fixação de sessão, cookies/tokens, account takeover.	POST-SessionHijack , AUTH-SessionFixation , POST-AccountTakeover , WS-WebSocketHijack
XVI	A Torre	torre	técnica	Comprometimento crítico: RCE, quebra de isolamento, controle do sistema de lab.	POST-RCE , INJ-CmdInjection , INJ-SSTI , INTEG-InsecureDeser , INTEG-FileUpload
XVII	A Estrela	estrela	técnica	Certificate Transparency: subdomínios, serviços TLS, novos pontos da infra.	RECON-CertTransparency , CRYPTO-TLSAnalysis
XVIII	A Lua	lua	técnica	Manipulação/ilusão/instruções ocultas: prompt injection, vazamento de system prompt, apps com LLM.	LLM-PromptInjection , LLM-SystemPromptLeak , LLM-InsecureOutput , LLM-ExcessiveAgency
XIX	O Sol	sol	sistema	❖ Consolidação da superfície (visão do campo) : reúne hosts, portas, tecnologias, endpoints e params; ilumina as próximas cartas jogáveis. É o expand_options visualizado.	(aciona recomputação de expand_options , 2.7)

#	Arcano	Ícone	Tipo	Função no jogo	Nós do grafo cobertos
XX	O Julgamento	juízo	sistema	Transforma descoberta em achado formal: evidência + técnica + impacto + severidade + referência + correção.	REPORT-Finding
XXI	O Mundo	mundo	sistema	Conclui a partida: aciona o Cronista, registra o caminho no grafo, calcula cobertura/custo, declara objetivo.	(aciona o Cronista final, 2.5)

Injeções web "clássicas" (SQLi, XSS, NoSQLi, XXE, LDAPi, GraphQL) ficam sob **O Mago** (são o ato de "conjurar" uma ferramenta contra um alvo) com selo da técnica; SSRF e IDOR/BAC ficam sob **Os Enamorados/O Imperador** conforme o eixo (confiança vs. privilégio). Essa distribuição é só a etiqueta `arcana` — a lógica continua nos nós do grafo.

5.3 Como 22 sprites cobrem ~90 nós (mudança no schema)

Adicione um campo `arcana` ao schema da carta (2.3). O `Card.tsx` passa a montar o sprite pelo **arcano**, não pelo `card_id`:

```
# em cada config/cards/<id>.yaml
id: INJ-SQLi
arcana: mago           # ← novo: qual Arcano Maior representa esta técnica
badge: "SQLi"          # ← rótulo curto sobreposto ao sprite do arcano
# ... resto do schema igual (2.3)
```

`ui/public/sprites/arcana/{arcana_id}.png` guarda as 22 artes. A UI renderiza: sprite do arcano + moldura + `badge` da técnica + o alvo concreto da carta-opção (ex: "SQLi · /rest/products?q"). Assim uma carta-opção de SQLi contra dois endpoints diferentes usa a mesma arte de "O Mago", com badges/alvos distintos.

5.4 Prompt-base para o Codex (gerar os 22 arcanos)

Ilustração de carta de TARÔ — Arcano Maior, formato retrato 3:4. Moldura ornamentada com numeral romano no topo e faixa de nome na base. Estilo inspirado na tradição Rider-Waite-Smith mas TOTALMENTE ORIGINAL (não reproduza ilustrações de baralhos existentes). Linha de contorno firme, simbolismo denso, paleta rica: dourado/ocre, azul-noite profundo, vermelho-sangue como acento. Releitura cyber-esotérica: os símbolos clássicos do arcano se fundem com tecnologia/rede.

Arcano: "{NUMERO} — {NOME}" ({FUNCAO_NO_JOGO}).
 Elemento simbólico central: {SIMBOLO_CLASSICO} reinterpretado com {ELEMENTO TECH}.
 Fundo atmosférico coerente com o arcano. SEM texto real na arte (numeral e nome são desenhados na moldura; badges de técnica vêm da UI depois).
 PNG 768x1024, alto contraste, adequado a exibição pequena.

Exemplos de fusão simbólica para preencher o prompt:

- **I — O Mago** (`mago`): a mesa com os quatro naipes vira um console; o cajado erguido é uma antena emitindo sinal; símbolo do infinito feito de fluxo de dados.
- **II — A Sacerdotisa** (`sacerdotisa`): véu entre duas colunas coberto de código-fonte; o pergaminho oculto é um arquivo `.env` vazando chaves.
- **IX — O Eremita** (`eremita`): a lanterna é um terminal brilhando no escuro; o cajado é um cabo de rede; passiva, silenciosa.
- **XVI — A Torre** (`torre`): a torre é um rack de servidores atingido por um raio, faíscas de pacotes caindo.
- **XVIII — A Lua** (`lua`): duas torres-firewall, um caminho de tokens; a lua tem um rosto que sussurra um prompt oculto; reflexos ilusórios.
- **XXI — O Mundo** (`mundo`): a guirlanda é um grafo de nós fechando um ciclo; a figura central segura um relatório selado.

Dica: gere primeiro **O Mago** e **A Torre** (o arquétipo mais usado e o mais dramático). Fixe paleta e moldura neles, depois repita a fórmula para os outros 20 — garante um baralho coeso.

5.5 Cartas-sistema: o que precisa ser implementado para o tarô funcionar

Cinco arcanos não são "ataques" — são **mecânicas da engine**. Alguns já existiam; A Justiça é novo e precisa ser construído. Isto é o "implemente o que falta":

- **O Louco (0) — semente.** Já coberto: é o estado inicial do `Field` + a primeira carta-opção (`RECON-PassiveDNS`) que a engine oferece no turno 1. Nenhuma implementação nova; só marque-o como a carta de abertura.
- **O Mago (I) — arquétipo do operador.** Já coberto pelo `ToolOperatorAgent` (2.12). Implementação nova = só visual: toda carta técnica usa a arte mago quando não tem arcano mais específico.
- **A Justiça (XI) — Agente Validador (NOVO, precisa implementar).** Um agente de sistema que roda **depois** de uma carta-técnica com achado. Ele: (a) re-testa a evidência de forma independente para confirmar (reduz falso-positivo), (b) consulta o `LOG-DetectionCheck` para ver se o alvo registrou/bloqueou a ação, (c) calcula a **severidade** (ex: CVSS simplificado) e a **confiança**, (d) marca o finding como `confirmed | unconfirmed | false_positive`. Só findings `confirmed` viram carta-opção de `0 Julgamento`. Isso eleva muito a qualidade científica (o paper "Hackers or Hallucinators?" da Parte 12 é exatamente sobre falso-positivo em pentest por LLM — A Justiça é sua resposta de projeto a isso). Vive em `agents/validate_finding.py` + nó `VALIDATE-Finding` no grafo. **Confirmação forte de vulns cegas via Interactsh** (2.18-Rodada 4): se o achado gerou um callback OOB registrado, A Justiça marca `confirmed` com evidência irrefutável. **Validação cruzada via ZAP-AF:** um segundo motor independente reexecuta o teste; concordância vira métrica de confiança para o artigo.
- **A Morte (XIII) — transição de fase.** Mecânica na engine: quando um ramo de recon se esgota ou uma vuln é `confirmed`, A Morte "mata" o ramo antigo e **promove** o estado para a próxima fase (recon→exploração→pós). Implementar como uma regra em `game_engine.apply_effects` que emite o evento `PhaseTransition` e recomputa `expand_options`.
- **O Sol (XIX) — consolidação/visão.** Mecânica: recomputa e **exibe** a superfície de ataque inteira (é o `expand_options` + um resumo do `Field`). Serve para o jogador "ver tudo" a qualquer momento. Implementar como um comando/carta que dispara `expand_options` e destaca as opções de maior valor.
- **O Julgamento (XX) e O Mundo (XXI).** Já cobertos: `REPORT-Finding` (por achado) e o Cronista final (2.5). Só precisam do skin de tarô.

Regra de ouro do fluxo tarô: **técnica → A Justiça (valida) → O Julgamento (formaliza) → ... → O Mundo (fecha)**. A Morte e O Sol atuam entre turnos (transição e visão). Isso dá ao jogo um arco narrativo que espelha 1:1 o método real de pentest.

PARTE 6 — Sprints de implementação (prompts prontos)

Trabalhe cada bloco comigo (fable) no VSCode **na ordem**, validando (rodando os testes) antes do próximo. Cada prompt já traz contexto suficiente; com este guia no repositório, referencie-o. Os sprints seguem a Parte 0.1: constroem de baixo para cima (contratos e engine primeiro, UI por último), então cada camada testa contra uma fronteira já estável — é o que evita bug de integração.

Sprint 0 — Fundação e regras

Contexto: estamos construindo o "PentestArena", um pipeline de pentest orquestrado por agentes, apresentado como um jogo de cartas de tarô. Leia `ARQUITETURA_PENTEST_CARD_GAME.md` na raiz — em especial a Parte 0.1 (contratos) — para o conceito e as fronteiras entre módulos. Vamos construir por sprints, de baixo para cima; não pule etapas.

Tarefa do Sprint 0:

1. Crie a estrutura de pastas da Parte 3 (vazias com `__init__.py` onde for pacote).
2. Crie `ETHICS.md` e `config/allowlist.yaml` com apenas: `localhost`, `127.0.0.1`, e um placeholder comentado. O orquestrador NUNCA deve agir fora dessa lista.
3. Crie `pyproject.toml` (Python 3.11, deps: `fastapi`, `uvicorn`, `httpx`, `pyyaml`, `jinja2`, `pytest`, `websockets`, `python-nmap`).
4. Crie `labs/docker-compose.yml` subindo OWASP Juice Shop em `localhost:3000`.
5. Escreva o schema da Carta (Parte 2.3) como `dataclass/pydantic` em `orchestrator/registry.py`, com `AUTO-DESCOBERTA`: scan de `cards/**/card.yaml` (modelo `card-package` da Parte 2.17). Crie 1 pasta-carta de exemplo (`cards/mago/A1-sql-injection/` com `card.md` + `card.yaml` + `payloads/.gitkeep`) para validar o loader. Não implemente agentes ainda. Ao final, mostre a árvore criada.

Sprint 1 — Estado, eventos e engine (o coração)

Sprint 1 — Game Engine, base de conhecimento e event sourcing.

0. config/knowledge/attack_graph.yaml + engine/knowledge.py: modele o attack graph EXPANDIDO (seção 2.10, ~60 nós; comece transcrevendo o grafo inteiro — é só dado) como nós (cartas-tipo, com id ATT&CK e categoria OWASP:2025 e fase) + arestas `enables`/`requires`. knowledge.py deve validar que é um DAG coerente e oferecer children_of(node), phase_of(node) e reachable_from(satisfied_nodes). As preconditions/unlocks das cartas DEVEM ser derivadas deste grafo (seção 2.8), não escritas soltas. NÃO implemente os agentes ainda — só o grafo e as consultas.
 1. engine/events.py: tipos de evento (CardPlayed, FindingAdded, StateRevealed, TurnStarted) e um EventLog append-only em JSONL.
 2. engine/field.py: classe Field que representa o estado do alvo (portas, tecnologias, forms/params, vulns reveladas). Deve ser RECONSTRUÍVEL a partir do EventLog (event sourcing).
 3. engine/game_engine.py:
 - turnos, orçamento (mana) global,
 - MOTOR DE COMPLETAÇÃO expand_options(field) (ver seção 2.7): enumera as entidades descobertas no campo (hosts, portas, endpoints, params, vulns), avalia as preconditions de cada carta-tipo e INSTANCIA uma carta-opção por entidade compatível (ex: uma SQLi por parâmetro injetável). Deve ser PURO e DETERMINÍSTICO dado o Field.
 - método apply_effects(result) que atualiza o Field a partir do state_delta e, em seguida, re-roda expand_options para revelar as novas cartas-opção.
 - NÃO-DETERMINISMO (2.16): arestas do grafo carregam `when` (condição) e `weight` (peso); expand_options avalia `when` e ordena/amostra por `weight` usando uma SEED gravada no EventLog. Mesma seed + mesmo log ⇒ partida idêntica (reprodutibilidade).
 4. tests/test_engine_hand.py: dado um Field mockado, valide que (a) só as cartas-tipo com preconditions satisfeitas geram opções, (b) uma carta-tipo gera N cartas-opção para N entidades compatíveis, (c) aplicar um state_delta que revela um novo endpoint faz surgir novas opções. Rode os testes e mostre verde.
- Nada de rede real aqui — só lógica pura e testável.

Sprint 2 — Contrato de agente + 2 agentes reais (mock-first)

Sprint 2 — Agentes-carta + operador de ferramenta.

1. cards/_base/base.py: implemente CardAgent E ToolOperatorAgent (seções 2.12/2.13), com o loop de operação de ferramenta (selecionar-construir-chechar allowlist+VPN+timeout-executar-parsear-follow-up-AgentResult). config/tools.yaml com o registro de ferramentas e o mapa nó-ferramenta. Toda metadata de agente carrega attack_id (ATT&CK) e owasp_id (OWASP) — obrigatório.
 2. Implemente 2 pastas-carta completas (Parte 2.17): cards/louco/H1-recon-seed/ (opera `nmap`) e cards/imperatriz/H4-web-crawl/ (opera `httpx`) — cada uma com card.md + card.yaml + agent.py (subclasse curta do molde 2.13) + payloads/ + tests/. Ferramenta ausente → status "tool_missing".
 3. Cada agente devolve state_delta que revela info e desbloqueia cartas conforme arestas do grafo.
 4. tests/test_agents_mock.py: teste os agentes contra um alvo MOCK (monkeypatch da rede), sem tocar a internet. Valide o formato do AgentResult e o comportamento da allowlist.
- Rode os testes. Depois, num passo separado, rode recon_ports de verdade contra localhost:3000 (com o Juice Shop no ar) e mostre a saída.

Sprint 3 — Orquestrador + loop de turno

Sprint 3 — Orquestrador.

1. orchestrator/orchestrator.py: recebe "jogar card_id", carrega o agente, aplica allowlist e orçamento, executa, grava eventos no EventLog e chama engine.apply_effects.
2. Um loop de jogo mínimo em CLI (scripts/play_cli.py): mostra o Field, mostra a mão jogável, deixa escolher uma carta (ou uma política automática: --auto greedy), executa, repete até acabar orçamento ou não haver cartas.
3. Demonstre uma partida completa contra localhost:3000 começando só com RECON-Portas na mão, e mostrando novas cartas (WEB-*) aparecendo conforme o recon revela endpoints.

Sprint 4 — Cronista (relatório)

Sprint 4 — Cronista (relatório PROGRESSIVO + trilha de ataque + export).

1. chronicler/chronicler.py: escuta o EventLog e ATUALIZA o relatório A CADA JOGADA (não só no fim) — mantém um "relatório vivo" consultável durante a partida (2.5).
2. Trilha de ataque por achado: reconstrua a partir do grafo+log a SEQUÊNCIA de cartas que levou até cada falha (0 Louco → ... → exploração → A Justiça), com link para a evidência tipada e âncora ATT&CK/OWASP de cada passo. É o "como chegou até a falha".
3. Template chronicler/templates/report.(md|html).j2: resumo executivo, escopo/ética, TRILHA por achado, achados por severidade, cobertura, cartas jogadas vs disponíveis, custo, recomendações, apêndice reprodutível (seed + hash do EventLog).
4. Export: Markdown → PDF (skill de PDF) e HTML; endpoint/botão "Baixar relatório".
5. Regra crítica: os FATOS vêm do log, nunca inventados. LLM só para redigir prosa sobre fatos já registrados.
6. Gere o relatório da partida do Sprint 3 (com pelo menos uma trilha completa) e salve em reports/.

Sprint 5 — API + Frontend do jogo

Sprint 5 – UI do jogo (as 3 zonas da Parte 2.19).

1. `api/server.py` (FastAPI): REST para estado + WebSocket que transmite ao vivo `TurnStarted`, `CardPlayed`, `FindingAdded` e – importante – os `ActivityEvent` e `Artifacts` tipados do agente (2.19).
2. `ui/` (React+Vite+TS):
 - `Board.tsx` (ZONA A): o campo como GRAFO vivo do alvo (nós = `hosts/portas/endpoints/params/vulns` que crescem); boss com "vida"=superfície restante; clicar num nó destaca as cartas que o miram.
 - `Hand.tsx`: cartas jogáveis; clicar = `PREVIEW` (ferramenta/alvo/custo/efeito) → confirmar joga.
 - `Card.tsx`: sprite do ARCANO + badge da técnica + alvo (5.3); placeholder cinza por ora.
 - `Inspector.tsx` (ZONA B): activity stream ao vivo do agente + `ArtifactViewer.tsx` que despacha por type (`log/table/http_exchange/screenshot/graph/json/file/diff/metric/finding` – 2.19) + veredito de A Justiça.
 - `Terminal.tsx` + `EventFeed.tsx` (ZONA C): terminal-mapa e feed cronológico global.
 - `ReportPanel.tsx`: o "relatório vivo" (2.5) crescendo a cada jogada + botão "Baixar relatório" (PDF/HTML) que chama o Cronista.
 - `ws.ts`: conecta no WebSocket e anima cada jogada nas 3 zonas.
3. Rode backend+frontend e mostre uma partida jogável no navegador contra `localhost:3000`, com pelo menos 2 tipos de artefato renderizados (ex: `table` do `nmap` e `http_exchange` do `SQLi`).

Sprint 5.5 — Terminal Linux embutido (o "mapa")

Sprint 5.5 – Terminal como mapa (ver seção 2.6).

1. `api/terminal.py`: abra um PTY Linux por sessão com o módulo ``pty`` do Python, rodando um shell DENTRO de um container Linux do `docker-compose` (nunca o host). Exponha um WebSocket `/ws/terminal` que faça o duplex: teclas → PTY, saída do PTY → cliente.
2. Wrapper de comandos: intercepte comandos do JOGO (``map``, ``hand``, ``play <card_id>``, ``inspect boss``, ``report``) e traduza para chamadas ao orquestrador/engine, imprimindo resposta formatada estilo shell. Comandos Linux normais (`ls`, `cat`, `nmap`, `curl`) passam ao PTY, MAS bloqueie qualquer alvo fora de `config/allowlist.yaml` antes de executar.
3. `ui/src/Terminal.tsx`: monte o `xterm.js` conectado ao `/ws/terminal`. Layout: board+mão de um lado, terminal do outro (ou painel inferior). As duas vistas compartilham o MESMO estado – jogar pelo board reflete no terminal e vice-versa.
4. O terminal deve ecoar cada jogada como um log de exploração (ex: "[+] novo caminho: `/rest/products`").
5. Segurança: container sem privilégios, rede só para alvos de lab. Prove com um teste que um comando para host fora da `allowlist` é recusado. Rode e mostre o terminal navegando uma partida.

Sprint 5.6 — VM isolada + botão de VPN (kill-switch)

Sprint 5.6 – Máquina de ataque isolada + VPN de controle (ver Parte 8 do doc).

1. `vm/Dockerfile.attack-box`: consolide backend + agentes + PTY do terminal numa única imagem de "máquina de ataque" isolada. Rede do container só deve alcançar o range de laboratório.
 2. `api/vpn.py`: implemente `connect()/disconnect()/status()` usando `WireGuard` (`wg-quick`) contra um serviço de gateway no `docker-compose` dos labs (simula a VPN mesmo em ambiente local). `status()` deve ler o estado REAL da interface, nunca uma flag em memória.
 3. Endpoints: `POST /vpn/toggle`, `GET /vpn/status`; emita o estado também no WebSocket geral.
 4. Kill-switch: um watcher em background detecta queda da interface VPN e publica evento `VpnDropped` no `EventLog`; o orquestrador deve então recusar qualquer carta em andamento ou nova.
 5. Orquestrador: antes de executar qualquer agente, checar `allowlist` E `vpn.status()=="connected"`. Se VPN estiver desligada, retornar status `"blocked_by_vpn_down"` (mesmo padrão de erro já usado para `allowlist`).
 6. `ui/src/VpnToggle.tsx`: botão verde/vermelho/amarelo no topo da UI, atualizado via WebSocket (não só na resposta do clique). Enquanto não estiver verde, desabilite visualmente as cartas que dependem de rede.
 7. `tests/test_vpn_gate.py`: prove que (a) carta é recusada com VPN desligada, (b) derrubar a VPN no meio de uma execução aciona o kill-switch e cancela a ação.
 8. `config/vpn/` deve entrar no `.gitignore` – nada de segredo real commitado.
- Rode os testes e demonstre: VPN desligada → cartas de rede bloqueadas na UI e no backend;
VPN ligada → jogo normal; derrubar a VPN no meio de uma jogada → kill-switch age.

Sprint 6 — Mais cartas + métricas para o artigo

Sprint 6 – Expansão e instrumentação.

1. Adicione agentes/cartas: `conn_tls`, `web_sql` (contra Juice Shop, payloads em config), `web_xss`. Cada um com `preconditions` corretas (ex: `sql` só entra na mão se `web_recon` achou `params`).
2. Instrumentação para o artigo: colete por partida – nº de jogadas, requisições por achado, cobertura (`endpoints/vulns` descobertos), tempo, e permita comparar políticas de escolha de carta (`random` vs `greedy` vs `llm-guided`). Salve métricas em CSV.
3. Um script de experimento que roda N partidas por política e agrega as métricas.

PARTE 7 — Ligação com o artigo científico

Estrutura sugerida do paper, mapeada no que o sistema já produz:

- **Introdução / problema:** pentest como pipeline rígido → proposta de orquestração adaptativa por estado via agentes componíveis.
- **Trabalhos relacionados:** scanners automáticos, frameworks de agentes, autonomia em segurança ofensiva.
- **Método (o núcleo):** a metáfora de cartas como modelo formal — Field (estado), Cartas (agentes com preconditions/effects), regra de mão jogável, orquestração, cronista. Diagramas = Parte 2.1 e o loop 1.2.
- **Implementação:** a stack e a estrutura (Partes 2–3).
- **Experimentos:** métricas do Sprint 6; comparação de políticas; reprodutibilidade via event log.
- **Discussão/limitações:** allowlist, alvos de lab, escopo, riscos de autonomia.
- **Ética:** ETHICS.md + a dupla trava técnica allowlist + VPN/kill-switch (Parte 8).

Contribuições declaráveis: (1) formalização "estado→cartas jogáveis" para pentest adaptativo; (2) arquitetura de agentes-carta componíveis com contrato padronizado; (3) documentação/reprodutibilidade por event sourcing; (4) camada de jogo como interface de explicação; (5) isolamento operacional via máquina de ataque + VPN com kill-switch como controle técnico redundante à allowlist.

PARTE 8 — A aplicação inteira é (na prática) uma Máquina Virtual + botão de VPN

Você percebeu certo: a soma de tudo o que descrevemos (terminal Linux embutido, agentes rodando ferramentas de rede, containers de laboratório) já É, na prática, uma **máquina de ataque isolada** — o mesmo conceito que um pentester usa numa VM dedicada (tipo Kali) para nunca testar a partir da própria máquina. Esta parte formaliza isso e explica **cada implementação com cuidado**, porque é aqui que segurança de verdade (sua e do alvo) é decidida.

8.1 Por que isso precisa ser uma "VM" de verdade (não só conceito)

Um operador de pentest nunca ataca a partir do próprio notebook: ele usa uma **máquina de ataque isolada**, geralmente conectada à rede-alvo só através de uma **VPN dedicada**. Replicamos exatamente esse modelo, por três motivos:

1. **Contenção:** se um agente (ou um comando digitado no terminal-mapa) tiver um bug ou comportamento inesperado, ele fica preso dentro da VM/container — nunca toca o sistema operacional do usuário.
2. **Rede controlada:** a VM só enxerga o alvo através do túnel VPN. Sem VPN ligada, não há rota de rede até o alvo — é uma segunda trava de segurança, **independente** da allowlist (defesa em profundidade).
3. **Fidelidade científica:** para o artigo, isso reproduz fielmente o setup real de um teste de intrusão profissional (attack box + VPN de acesso ao range), o que fortalece a seção de metodologia.

8.2 Duas formas de implementar a "VM" — escolha uma

Opção	Como	Quando usar
A — Container Docker como "VM lógica" (recomendado para começar)	Um único container Linux (<code>vm/Dockerfile.attack-box</code>) roda o backend, os agentes, o PTY do terminal e o cliente VPN. É leve, sobe em segundos, e já cobre 100% do isolamento de rede necessário para o TCC/artigo.	Desenvolvimento, demonstrações, a maior parte do artigo. É o que os sprints abaixo implementam.
B — VM completa (VirtualBox/QEMU/Vagrant com imagem tipo Kali)	Uma VM de verdade, com kernel e SO próprios, rodando o mesmo backend dentro dela.	Se quiser máxima fidelidade ("é literalmente uma VM") para a defesa/banca, ou se algum agente precisar de recursos que Docker não isola bem (ex: manipulação de interface de rede raw).

Recomendação: **implemente A primeiro** (é o que já está nos sprints). Documente no artigo que a arquitetura é "portável para uma VM completa (opção B) sem mudar a lógica do orquestrador", já que o backend não sabe se está num container ou numa VM — só enxerga "estou isolado, tenho uma interface de rede que só funciona com VPN ligada".

8.3 O botão de VPN — comportamento exato

Um botão fixo no topo da UI (ao lado do terminal), com três estados visuais:

- **VERDE Verde — Conectado:** o túnel VPN está ativo; a VM tem rota até a rede do alvo. Cartas que dependem de rede podem ser jogadas.
- **VERMELHO Vermelho — Desconectado:** sem túnel; qualquer carta que precise de rede é bloqueada pelo orquestrador antes de rodar (mesmo texto de erro que já existe para allowlist: `"blocked_by_allowlist"` vira, aqui, `"blocked_by_vpn_down"`).
- **AMARELO Amarelo — Conectando/instável:** transição, ou o daemon caiu inesperadamente no meio da partida (kill-switch, ver 8.4).

O botão **não é decorativo** — ele reflete e controla o estado real de um processo VPN rodando dentro da VM/container.

8.4 Implementação passo a passo

Backend — `api/vpn.py`: 1. Escolha o protocolo: **WireGuard** (`wg-quick up/down`, mais simples de automatizar) ou **OpenVPN** (`.ovpn`, comum em labs estilo HTB/TryHackMe). Para labs locais (Juice Shop/DVWA em Docker), a "VPN" pode ser simulada como uma rede Docker isolada com um serviço WireGuard fazendo de gateway — assim você já nasce com o hábito certo, mesmo em laboratório local. 2. `vpn.connect()` / `vpn.disconnect()`: chamam o binário via subprocess dentro do container, lendo a config de `config/vpn/` (nunca hardcoded). 3. `vpn.status()`: consulta o estado real da interface (`wg show` ou parse do log do OpenVPN) — nunca confie só numa flag em memória, sempre confirme com o processo/interface de verdade. 4. Endpoint REST `POST /vpn/toggle` + `GET /vpn/status`, e emita o estado também no WebSocket geral, para o botão atualizar ao vivo. 5. **Kill-switch:** um watcher em background que, se a interface VPN cair no meio de uma partida, dispara imediatamente um evento `VpnDropped` no EventLog e o orquestrador **cancela/recusa qualquer carta em andamento ou nova** até a VPN voltar. Isso é análogo ao kill-switch de qualquer VPN de segurança séria.

Orquestrador — mudança no fluxo de "jogar carta":

```
jogar carta → checar allowlist do alvo → checar vpn.status() == "connected"
→ se qualquer checagem falhar: status = "blocked_by_allowlist" | "blocked_by_vpn_down"
→ senão: executa o agente normalmente
```

Isso é literalmente uma linha a mais de defesa antes da que já existia (Parte 2.3/Sprint 2) — **não substitui a allowlist, soma-se a ela.**

Frontend — `ui/src/VpnToggle.tsx`: - Botão que chama `POST /vpn/toggle` e escuta o WebSocket para atualizar a cor em tempo real (não confie só na resposta do clique — o estado pode mudar por conta própria, ex. kill-switch). - Enquanto vermelho/amarelo, desabilite visualmente (não apenas no backend) as cartas que exigem rede, para o usuário entender o motivo antes de tentar jogar.

Segurança de configuração: - `config/vpn/` inteiro vai para o `.gitignore` — nenhum `.ovpn`, chave ou certificado é versionado. Documente isso em `ETHICS.md`. - A VM/container só deve ter rota de rede para: (a) o range de laboratório via VPN, (b) o necessário para baixar dependências. Nunca acesso irrestrito à internet a partir da VM de ataque.

8.5 O que colocar no artigo sobre isso

Trate o par "**máquina de ataque isolada + VPN com kill-switch**" como uma **segunda camada de controle técnico**, independente e redundante à allowlist de software. Argumento para a seção de metodologia/ética: mesmo que a lógica de allowlist tivesse um bug, a ausência de rota de rede (VPN desligada) impede fisicamente qualquer ação contra um alvo — dois mecanismos independentes reduzem a chance de uso indevido a partir de uma única falha.

PARTE 9 — Checklist de validação (fechar cada item durante a implementação)

- [] Allowlist bloqueia alvo não autorizado (teste automatizado prova isso).
- [] Attack graph carregado de `attack_graph.yaml`; preconditions/unlocks das cartas derivam dele (não soltas).
- [] `expand_options` gera cartas-opção por entidade descoberta e é determinístico dado o Field (teste prova).
- [] Jogar uma carta muda o Field e faz surgirem novas cartas-opção automaticamente (teste do ciclo 2.7).
- [] Terminal-mapa autocompleta (`play` +TAB) exatamente as cartas-opção disponíveis.

- [] Deck vem PRÉ-CONFIGURADO: cada carta em `config/cards/` já tem ferramenta, `payload_set`, limites, arcano e âncora — jogável sem configuração manual.
- [] Arestas com `when / weight` produzem ramos diferentes conforme o achado (árvore não-determinística, 2.16).
- [] Mesma seed + mesmo EventLog re-executam uma partida idêntica (não-determinismo reprodutível).
- [] Field é reconstruível 100% a partir do EventLog (teste de round-trip).
- [] Todo AgentResult segue o schema 2.3 (validação).
- [] Todo agente carrega `attack_id` (ATT&CK) e `owasp_id` (OWASP) na metadata.
- [] ToolOperatorAgent opera a ferramenta certa do `tools.yaml`; ferramenta ausente → `tool_missing`.
- [] Cada carta é uma pasta autocontida (`card.md` + `card.yaml` + `agent.py` + `payloads/` + `tests/`) e o registry auto-descobre `cards/**/card.yaml`.
- [] A VM de ataque (`Dockerfile.attack-box`) instala o toolkit completo; toda carta encontra sua ferramenta.
- [] Subir arquivos em `payloads/` de uma carta é usado pelo agente na jogada seguinte (sem re-registrar).
- [] Ferramenta com `net:true` só executa com a VPN ligada (teste prova).
- [] Cada carta tem `arcana` válido (um dos 22) e a UI resolve o sprite por `arcana_id` + `badge`.
- [] Agente Validador (A Justiça) marca findings como `confirmed/unconfirmed/false_positive`; só `confirmed` chega a `REPORT-Finding`.
- [] Relatório do cronista usa apenas fatos do log (sem números inventados).
- [] Relatório é construído PROGRESSIVAMENTE (atualiza a cada jogada) e visível ao vivo na UI.
- [] Cada achado tem a TRILHA de ataque reconstruída (sequência de cartas do início até a falha).
- [] Relatório final é baixável em PDF e HTML, formatado, com evidências embutidas.
- [] Partida ao vivo no navegador mostra cartas surgindo conforme o recon avança.
- [] Agente emite activity stream + artefatos tipados; a UI renderiza cada tipo no visualizador certo (2.19).
- [] Pelo menos `table`, `http_exchange` e `finding` renderizam corretamente no Inspector.
- [] Sprints rodam contra Juice Shop local, nunca contra alvo externo.
- [] Botão de VPN reflete o estado real da interface (não uma flag solta em memória).
- [] Carta com dependência de rede é recusada com `blocked_by_vpn_down` quando a VPN está desligada.
- [] Kill-switch: derrubar a VPN em teste automatizado durante uma "partida" cancela a ação em andamento.
- [] `config/vpn/` está no `.gitignore` e nenhum segredo de VPN foi commitado.
- [] Forja cria uma carta válida (YAML + esqueleto de agente) e ela vira jogável quando as preconditions batem.
- [] Partidas persistem no SQLite e podem ser recarregadas/reproduzidas a partir do EventLog.
- [] Agente respeita timeout e limite de requisições; falha vira `status:error` sem derrubar a partida.
- [] Teste e2e sobe os labs, joga uma partida e valida os achados no relatório final.
- [] Binário congelado roda num ambiente limpo (sem Python) e acha todos os recursos embutidos.
- [] Binário de demo roda SEM Docker (alvo-mock embutido); imagem reproduzível roda COM Docker.
- [] Nenhum segredo (API/VPN) embutido no binário — pedido no 1º uso e guardado no keyring.

PARTE 10 — Análise de lacunas: o que ainda falta implementar

Fiz uma passada crítica no projeto inteiro. O que já está especificado cobre o *fluxo de jogo* (campo → cartas → agentes → cronista) e a *segurança* (allowlist + VM + VPN). Mas há peças que, se não forem tratadas agora, vão travar o projeto — principalmente porque no final tudo vira um **binário distribuível**. Abaixo, cada lacuna e a solução.

10.1 A "Forja de Cartas" (o ponto que mais faltava)

Você descreveu o coração da pesquisa: "*eu configuro o agente previamente e ele vira uma nova carta*". Isso **ainda não tinha um fluxo** — as cartas estavam sendo tratadas como arquivos YAML escritos à mão. Precisa existir um módulo que transforma "um agente configurado" numa carta jogável.

Solução — módulo `forge/` : - Uma interface (aba na UI + comando `forge` no terminal-mapa) onde você define: qual ferramenta/técnica o agente usa, seus parâmetros (`config`), as `preconditions` (quando entra na mão), os `effects` (o que revela/desbloqueia), custo e risco. - Ao salvar, a Forja **valida** contra o schema 2.3, **gera o `config/cards/<id>.yaml`** e, se for um agente novo, cria o esqueleto `agents/<id>.py` a partir de um template (`forge/templates/agent.py.j2`). - Isso torna o sistema **extensível sem editar código na mão** — central para o argumento científico de "capacidades componíveis". Cada carta forjada é versionada e reproduzível. - Para o artigo: a Forja é a evidência de que o pipeline é *aberto* — qualquer nova técnica de pentest vira uma carta seguindo o mesmo contrato.

10.2 Persistência e estado entre sessões

Hoje só existe o EventLog em JSONL (bom para reprodutibilidade, ruim para consulta). Falta guardar partidas, decks e cartas forjadas de forma consultável.

Solução — SQLite embutido (`storage/db.py`): uma tabela de partidas (id, alvo, timestamp, resultado, caminho do EventLog), uma de cartas forjadas, uma de métricas. SQLite é ideal porque é um **único arquivo, sem servidor** — casa perfeitamente com a meta de virar binário. O EventLog continua sendo a fonte da verdade para reconstruir o Field; o SQLite é o índice consultável por cima.

10.3 Integração com LLM (agentes `llm_assisted`)

Várias cartas têm `llm_assisted: true`, mas não estava definido **como** o Claude é chamado nem onde a chave da API vive.

Solução — `llm/client.py`: um wrapper único sobre a API Claude com (a) chave lida de variável de ambiente / keyring do SO (nunca hardcoded, nunca no binário), (b) templates de prompt por tipo de carta em `llm/prompts/`, (c) rate limiting e teto de custo por partida, (d) modo *offline/mock* para rodar experimentos sem gastar API (essencial para reprodutibilidade e para o binário funcionar sem chave). Regra de ouro que já vale para o cronista: **o LLM adapta payloads/prosa, mas decisões de segurança (allowlist/VPN) nunca dependem dele.**

10.4 Robustez de execução dos agentes

Falta o tratamento do mundo real: um `nmap` que trava, um alvo que não responde, um agente que estoura tempo.

Solução: todo agente roda com **timeout** e **limite de requisições** (já previsto em `config.max_requests`); falhas viram `status: "error"` com a exceção capturada no EventLog, nunca derrubam a partida; execução de um agente por vez com **lock de turno** (evita corrida de estado no Field). Opcional: fila assíncrona se quiser paralelizar no futuro — mas comece serial, é mais fácil de raciocinar e defender no artigo.

10.5 Objetivos, vitória e economia de mana

O "boss" tem vida, mas as condições de fim de jogo estavam vagas.

Solução — `config/game_rules.yaml`: defina objetivo (ex: "revelar $\geq N$ vulnerabilidades" ou "esgotar a superfície mapeável"), condição de derrota (orçamento/mana zerado sem objetivo), e a economia de custo por categoria de carta. Deixar isso em config torna fácil rodar variações para os experimentos do artigo.

10.6 Configuração e segredos

Chaves de API, perfis de VPN, endpoints — nada disso pode estar no código nem no binário.

Solução: um `.env` para dev + leitura via keyring do SO em produção; um `config/settings.py` (pydantic-settings) centraliza tudo. `.gitignore` cobre `.env`, `config/vpn/*`, `*.key`. No binário, segredos são pedidos ao usuário no primeiro uso e guardados no keyring, **nunca embutidos**.

10.7 Cross-platform (a lacuna que morde na hora do binário)

O projeto usa PTY (`pty` só existe em Unix), WireGuard e Docker — todos com comportamento diferente em Windows/macOS/Linux.

Solução — assumir Linux como alvo primário e isolar o resto: - O backend inteiro roda **dentro da VM/container Linux** (Parte 8), então PTY, WireGuard e ferramentas de pentest sempre executam em Linux, independentemente do SO do usuário. - O que roda no SO do usuário é só o *shell* de UI (navegador ou app desktop) + o launcher que sobe o container. Isso resolve 90% do problema de portabilidade de uma vez, e é o que viabiliza o binário (ver Parte 11).

10.8 Testes de integração e ponta-a-ponta

Os sprints têm testes unitários, mas falta o teste que exercita o fluxo inteiro.

Solução: um teste e2e que sobe os labs, joga uma partida scriptada contra o Juice Shop, e valida que o relatório final contém os achados esperados. Rode-o no CI. É também a sua "demo reproduzível" para a banca.

10.9 Resumo das lacunas → novos artefatos

Lacuna	Onde resolve
Forja de cartas (agente→carta)	forge/ + Sprint 7
Persistência consultável	storage/db.py (SQLite) + Sprint 7
Integração LLM	llm/client.py + llm/prompts/
Robustez de agentes	agents/base.py (timeouts/locks)
Regras de jogo	config/game_rules.yaml
Segredos/config	config/settings.py + keyring
Cross-platform	backend dentro da VM Linux
Teste e2e	tests/test_e2e.py + CI
Empacotamento em binário	Parte 11 + Sprint 8

PARTE 11 — Transformar o projeto num binário distribuível

Esta é a meta final e a mais delicada, porque o PentestArena **não é** um script simples: é frontend web + backend Python + container Linux + VPN. Um "binário único" ingênuo não consegue conter um daemon Docker. A solução é uma **estratégia em camadas** — seja honesto sobre isso no artigo, porque a decisão de empacotamento é em si uma contribuição de engenharia.

11.1 Primeiro, colapsar as camadas

1. **Frontend → dentro do backend:** faça `vite build` gerando assets estáticos e sirva-os pelo próprio FastAPI (`app.mount("/", StaticFiles(...))`). Assim não há dois servidores — só o backend Python serve a UI e as APIs. Uma coisa a empacotar em vez de duas.
2. **Backend Python → um executável:** use **PyInstaller** (mais maduro) ou **Nuitka** (compila para C, binário mais rápido e mais difícil de inspecionar) para congelar o backend + assets estáticos num único executável, sem exigir Python instalado na máquina do usuário.

11.2 O problema do Docker/VPN — três estratégias (escolha por objetivo)

Estratégia	Como	Trade-off
A — Binário + Docker como pré-requisito (recomendado)	O binário contém backend+UI+lógica; ao subir, ele orquestra o Docker do usuário (sobe labs, VM de ataque, gateway VPN via API do Docker). Documente "requer Docker instalado".	Não é 100% autocontido, mas é realista e mantém o isolamento de segurança intacto. É o caminho profissional.
B — Binário standalone com alvo embutido (para a demo/banca)	Empacote junto um alvo-mock leve em processo (um servidor vulnerável simulado em Python, sem Docker) e desligue os agentes que exigem ferramentas externas. O binário roda sozinho, clicou-abriu.	Perde fidelidade (não é o Juice Shop real), mas é a versão "roda em qualquer PC sem instalar nada" — ótima para apresentar.
C — Imagem única distribuível (Docker/OCI ou VM appliance)	Em vez de um <code>.exe</code> , distribua uma imagem Docker (<code>docker run pentest-arena</code>) ou uma VM appliance (<code>.ova</code>) com tudo dentro.	Tecnicamente é "um artefato", mas exige Docker/VirtualBox no destino. Máxima reprodutibilidade científica.

Recomendação prática: entregue **duas** saídas — a **B** como binário de demonstração (impressiona na defesa, roda sem setup) e a **C** (imagem/appliance) como o artefato reprodutível oficial do artigo. A **A** é o caminho se o projeto virar ferramenta de uso contínuo.

11.3 Opcional — shell desktop nativo

Se quiser uma janela de app em vez do navegador, envolva a UI com **Tauri** (leve, Rust, binário pequeno) ou **Electron** (mais pesado, mais conhecido). O backend Python (congelado por PyInstaller) roda como *sidecar process* que o Tauri/Electron sobe e conversa por localhost. Tauri é o mais alinhado com "binário enxuto".

11.4 Ordem de empacotamento (sem surpresas no fim)

1. Congele já no Sprint 8 um "hello world" do backend com PyInstaller — descubra cedo os problemas de import dinâmico (PyInstaller costuma perder imports feitos por string; declare-os em `hiddenimports`).
2. Garanta que **todo caminho de arquivo** (cards, sprites, templates) funcione tanto em dev quanto congelado — use `sys._MEIPASS` (PyInstaller) para achar recursos embutidos. Centralize isso num helper `resource_path()`.
3. Segredos **nunca** entram no binário: peça no primeiro uso, guarde no keyring.
4. Só empacote depois que o e2e (10.8) passar — não tente congelar um alvo em movimento.

11.5 Novos sprints

Sprint 7 — Forja de cartas + persistência

Sprint 7 — Forja e persistência (ver Parte 10.1 e 10.2).

1. `forge/`: um serviço que recebe a definição de um agente (arcano, ferramenta, params, preconditions, effects, custo, risco), valida contra o schema 2.3 e GERA A PASTA-CARTA INTEIRA em `cards/<arcano>/<id>/` (`card.md` + `card.yaml` + `agent.py` + `parser.py` + `payloads/` + `tests/`) a partir de `forge/templates/` (ver Parte 2.17). O registry auto-descobre a pasta nova no próximo boot.
2. UI: uma aba "Forja" + comando `forge` no terminal para criar a pasta-carta sem editar à mão.
3. `storage/db.py`: SQLite com tabelas de partidas, cartas forjadas e métricas. O EventLog continua fonte da verdade; o SQLite é índice consultável.
4. Ao fim de cada partida, grave o resumo no SQLite e permita recarregar/reproduzir partidas antigas.
5. Testes: forjar uma carta e comprovar que ela aparece jogável quando as preconditions batem.

Sprint 8 — Empacotamento em binário

Sprint 8 — Binário (ver Parte 11).

1. Faça `vite build` e sirva o frontend estático pelo FastAPI (uma porta só).
2. Centralize acesso a recursos num helper `resource_path()` compatível com PyInstaller (`sys._MEIPASS`).
3. Crie o build com PyInstaller (`--onefile`) gerando o executável do backend+UI. Declare `hiddenimports` para os agentes/cartas carregados dinamicamente. Teste o binário num ambiente limpo.
4. Implemente a estratégia B (alvo-mock embutido em Python) para o binário rodar SEM Docker, e deixe a estratégia A (orquestrar Docker do usuário) atrás de uma flag `--labs`.
5. Produza também a estratégia C: um `docker-compose/Dockerfile` que empacota tudo numa imagem única.
6. Segredos: no primeiro uso, peça chave de API/VPN e guarde no keyring do SO — nunca no binário.
7. Entregue: (a) binário de demo standalone, (b) imagem Docker reproduzível. Documente ambos no README.

PARTE 12 — Literatura e referências (o que colocar em `docs/literature/`)

Esta é a bibliografia para implementar com contexto correto, e para você citar no artigo. Organizei em quatro blocos, com uma marcação clara: **[BAIXAR]** = documento livre/gratuito, coloque o PDF/HTML de verdade dentro do repositório; **[CITAR]** = livro comercial, não distribua o arquivo (direitos autorais) — compre/adquira legalmente e apenas cite/referencie no artigo e no README.

Estrutura de pasta sugerida:

```
docs/literature/
├── 00-standards/      # ATT&CK, ATLAS, PTES, OWASP (todos [BAIXAR])
├── 01-papers-agentes/ # papers acadêmicos de agentes de IA em pentest ([BAIXAR], arXiv)
├── 02-papers-fundacao/ # ReAct, Toolformer, Reflexion — arquitetura de agente ([BAIXAR])
├── 03-livros/         # NÃO coloque os PDFs — só um BIBLIOGRAFIA.md citando ([CITAR])
├── 04-ferramentas/    # docs oficiais de nmap/sqlmap/ffuf/nuclei/etc. ([BAIXAR])
```

12.1 Padrões e frameworks oficiais (base do attack graph — Parte 2.8/2.10/2.11)

- **[BAIXAR]** MITRE ATT&CK — matriz de táticas/técnicas. <https://attack.mitre.org> — baixe as páginas das táticas usadas no grafo (TA0043 Reconnaissance, TA0001 Initial Access, TA0006 Credential Access, TA0008 Lateral Movement, TA0010 Exfiltration, etc.).
- **[BAIXAR]** MITRE ATLAS (Adversarial Threat Landscape for AI Systems) — irmão do ATT&CK, específico para ataques contra sistemas de IA (16 táticas, 84 técnicas, v5.1.0/nov-2025). Cobre exatamente a camada LLM (2.11): data

poisoning, model evasion, prompt injection, RAG poisoning. <https://atlas.mitre.org> — este é o framework que faltava citar formalmente ao lado do OWASP LLM Top 10.

- **[BAIXAR]** PTES (Penetration Testing Execution Standard) — as 7 fases que estruturam as camadas do grafo (2.1/2.10). <http://www.pentest-standard.org>
- **[BAIXAR]** OWASP Top 10:2025 (web) — <https://owasp.org/Top10/2025/>
- **[BAIXAR]** OWASP API Security Top 10 (2023) — <https://owasp.org/API-Security/editions/2023/en/0x00-header/>
- **[BAIXAR]** OWASP Top 10 for LLM Applications (2025) — <https://genai.owasp.org/llm-top-10/>
- **[BAIXAR]** OWASP Web Security Testing Guide (WSTG) — metodologia detalhada de teste por categoria, complementa os `build_command` / `parse` dos agentes. <https://owasp.org/www-project-web-security-testing-guide/>
- **[BAIXAR]** NIST SP 800-115 — "Technical Guide to Information Security Testing and Assessment" (referência formal de metodologia, boa para a seção de métodos do artigo). <https://csrc.nist.gov/pubs/sp/800/115/final>

12.2 Papéis de pesquisa — agentes de IA aplicados a pentest (relacionados diretos da sua IC)

- **[BAIXAR]** Deng et al., "PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing" — USENIX Security 2024. O paper mais citado da área; ótimo ponto de partida para "trabalhos relacionados". <https://arxiv.org/abs/2308.06782>
- **[BAIXAR]** "PentestAgent: Incorporating LLM Agents to Automated Penetration Testing" — ACM AsiaCCS 2025. <https://arxiv.org/pdf/2411.05185>
- **[BAIXAR]** "VulnBot: Autonomous Penetration Testing for a Multi-Agent Collaborative Framework" (2025) — arquitetura multi-agente próxima da sua (orquestrador + agentes especializados por fase).
- **[BAIXAR]** "A Survey of LLM-Driven Penetration Testing: Taxonomy, Co-Evolution, and Open Challenges" (2025/2026) — survey recente, boa fonte de taxonomia e "open challenges" para a seção de limitações. <https://arxiv.org/pdf/2607.02605>
- **[BAIXAR]** "What Makes a Good LLM Agent for Real-world Penetration Testing?" (2026) — direto ao ponto de projeto de agente que você está resolvendo. <https://arxiv.org/html/2602.17622>
- **[BAIXAR]** "Hackers or Hallucinators? A Comprehensive Analysis of LLM-Based Automated Penetration Testing" (2026) — bom contraponto crítico/limitações (hallucination em achados) para sua seção de discussão.
- **[BAIXAR]** "Automated Penetration Testing with LLM Agents and Classical Planning" (2025) — conecta com a ideia do attack graph (planejamento clássico sobre um grafo de estados, muito próximo do seu `expand_options`).
- **[BAIXAR]** "LLM Agents Security Duality: A Comprehensive Survey of Self-Security and Empowered Cybersecurity" (2026) — relevante para o "loop reflexivo" que você mesmo notou (agente que testa IA usando riscos que também ameaçam agentes).
- Repositório de referência cruzada (não é paper, mas cataloga o campo): **LLM4Pentest** — <https://github.com/simon-p-j-r/LLM4Pentest>

12.3 Papéis fundacionais de arquitetura de agente (por que o `ToolOperatorAgent` funciona assim)

- **[BAIXAR]** Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models" (2022) — a base do loop pensar→agir→observar que o `ToolOperatorAgent` (2.12/2.13) implementa. <https://arxiv.org/abs/2210.03629>
- **[BAIXAR]** Schick et al., "Toolformer: Language Models Can Teach Themselves to Use Tools" (2023) — fundamenta a escolha de "o agente decide qual ferramenta e como invocá-la". <https://arxiv.org/abs/2302.04761>
- **[BAIXAR]** Shinn et al., "Reflexion: Language Agents with Verbal Reinforcement Learning" (2023) — relevante para o "follow-up" do loop 2.12 (o agente reavalia e tenta de novo com base no resultado anterior).

12.4 Livros de referência em pentest/segurança ofensiva (citar, não redistribuir)

- **[CITAR]** Stuttard & Pinto — *The Web Application Hacker's Handbook* (2ª ed.), Wiley. Referência clássica para os agentes `WEB-*` / `INJ-*` / `AUTH-*`.
- **[CITAR]** Georgia Weidman — *Penetration Testing: A Hands-On Introduction to Hacking*, No Starch Press. Boa base metodológica para as fases PTES.
- **[CITAR]** Peter Kim — *The Hacker Playbook 3: Practical Guide to Penetration Testing*, Secure Planet LLC.
- **[CITAR]** Justin Seitz & Tim Arnold — *Black Hat Python* (2ª ed.), No Starch Press. Útil para os `agents/*`.py que fazem parsing/automação.
- **[CITAR]** OccupyTheWeb — *Linux Basics for Hackers*, No Starch Press. Referência direta para o terminal-mapa (Parte 2.6) e a VM de ataque (Parte 8).

- **[CITAR]** Vickie Li — *Bug Bounty Bootcamp*, No Starch Press. Bom complemento para os nós de API/web modernos (2.11).
- **[CITAR]** Martin Kleppmann — *Designing Data-Intensive Applications*, O'Reilly. Fundamenta o event sourcing do EventLog/Field (2.5/10.2) — cite o capítulo de log-based architectures.

12.5 Documentação oficial das ferramentas (o que o ToolOperatorAgent de fato chama)

[BAIXAR] todas — são manuais/ `--help` oficiais, pequenos e valem estar versionados junto dos agentes que os usam: nmap (<https://nmap.org/book/>), httpx (projectdiscovery), ffuf (<https://github.com/ffuf/ffuf>), nuclei (projectdiscovery), sqlmap (<https://github.com/sqlmapproject/sqlmap/wiki>), dalfox, hydra, jwt_tool, graphw00f, garak (LLM red-teaming — <https://github.com/leondz/garak>), WireGuard (<https://www.wireguard.com/>), PyInstaller (<https://pyinstaller.org/en/stable/>).

12.6 Instrução de contexto para a implementação

Contexto adicional de literatura: em docs/literature/ (ver ARQUITETURA_PENTEST_CARD_GAME.md Parte 12) estão os documentos-fonte do attack graph e da arquitetura de agente. Ao implementar cada carta em cards/<arcano>/<id>/, cite a técnica ATT&CK/ATLAS e a categoria OWASP correspondente também num comentário no código, apontando para o arquivo em docs/literature/00-standards/ que a define. Ao implementar o loop do ToolOperatorAgent (2.12/2.13), trate-o como uma implementação enxuta do padrão ReAct (docs/literature/02-papers-fundacao/), documentando essa correspondência no docstring da classe.

PARTE 13 — As 12 interfaces da aplicação (mapa de telas)

A Arena (Parte 2.19) é só uma das telas. O aplicativo completo tem 12 interfaces. Abaixo cada uma, o que **consume** (dados já existentes) e o que exige de **backend novo**, mais a rota e a prioridade. Boa notícia: a maioria reaproveita sistemas já projetados — poucas pedem backend inédito.

#	Interface	Rota	Função	Consome	Backend novo?	Prio
1	Central de Comando	/command	Preparar a missão: escolher lab, objetivo, orçamento, política; checar VM/Docker/VPN/ tools; iniciar.	allowlist, game_rules, status dos serviços	⚙ endpoint de <i>preflight/status</i>	P1
2	Atlas do Ataque	/atlas	Ver o attack graph inteiro (~90 nós) com estados e filtros (arcano, OWASP/ATT&CK/ ATLAS/PTES, fase, severidade, ferramenta, caminho).	attack_graph + estados do Field	não (só leitura do grafo)	P1
3	Grimório	/grimoire	Compêndio das cartas: 22 arcanos, técnicas, ferramenta, pré-condições/efeitos, referências, taxa de sucesso/FP, exemplos de evidência.	registry (cards/) + métricas (SQLite)	não	P1
4	Laboratório de Alvos	/labs	Instalar/iniciar/pausar/restaurar Juice Shop/ DVWA/WebGoat; credenciais, URL, dificuldade, snapshot, saúde.	docker-compose dos labs	⚙ <i>Docker control API</i>	P3
5	Mesa de Evidências	/evidence	Investigar e correlacionar achados: req/resp, before/after, screenshots, JSON, marcação manual, cadeia de custódia (hash/hora/ carta).	artefatos tipados (2.19) + findings	não	P2
6	Linha do Tempo / Replay	/replay	Reproduzir a partida evento a evento; ver o campo em cada turno; exportar replay.	EventLog (event sourcing 2.5)	não	P1
7	Sala do Cronista	/report	Construir/revisar o relatório; preview do PDF; export MD/PDF/JSON; revisar prosa do LLM sem mudar fatos.	chronicler (Sprint 4)	não	P2
8	Observatório de Métricas	/metrics	Dashboards: cobertura, req/achado, achados por família, FP, caminhos no grafo, tempo/ custo por agente, ASR por política; comparar partidas.	métricas (Sprint 6, SQLite/CSV)	não	P2
9	Arena Comparativa	/compare	Rodar o mesmo cenário com várias políticas (humano/aleatória/gulosa/menor-custo/LLM/ playbook) e comparar.	experiment runner (Sprint 6)	⚙ <i>runner de experimentos</i>	P3
10	Sala da Justiça	/justice	Fila de validação: evidência suficiente?, duplicata?, confirmado/FP/inconclusivo, CVSS, reteste, validação cruzada.	agente Validador (5.5)	não	P2
11	Oráculo	/oracle	Recomendação explicável da próxima jogada pelo LLM: carta, alvo, evidências que motivaram, custo/risco, confiança, alternativas; aceitar/rejeitar.	política LLM (10.3)	⚙ <i>payload de explicabilidade</i>	P2
12	Centro de Segurança	/security (sempre acessível)	VPN, allowlist, interface de rede, kill-switch, agentes ativos, requisições/limites, alvo efetivo, botão de emergência, auditoria de bloqueios.	vpn.py + gates do orquestrador (Parte 8)	⚙ <i>feed de auditoria</i>	P1 (segurança)

Backend novo necessário (só 5 peças)

A maioria das telas é só leitura de dados já planejados. As que pedem código novo: 1. **Serviço de preflight/status** (Central de Comando + Centro de Segurança): checa VM, Docker, VPN, ferramentas instaladas e allowlist, e devolve um status consolidado. 2. **Docker control API** (Laboratório de Alvos): start/stop/restart/snapshot/health dos containers de lab, via SDK do Docker. 3. **Runner de experimentos** (Arena Comparativa): já previsto no Sprint 6 — executa N partidas por política e agrega. 4. **Payload de explicabilidade do LLM** (Oráculo): a política LLM já decide; falta expor *por que* (evidências, alternativas, confiança) num formato que a UI mostra. 5. **Feed de auditoria** (Centro de Segurança): stream das ações bloqueadas (allowlist/VPN) — já estão no EventLog, só falta uma view dedicada.

Navegação da aplicação

```
Central de Comando (/command)
├─ Laboratórios (/labs)
├─ Arena (/arena) → Campo · Inspetor · Terminal · Relatório-vivo · Centro de Segurança (overlay)
├─ Atlas do Ataque (/atlas)
├─ Grimório (/grimoire)
├─ Mesa de Evidências (/evidence)
├─ Sala da Justiça (/justice)
├─ Sala do Cronista (/report)
├─ Replay (/replay)
├─ Observatório (/metrics)
├─ Arena Comparativa (/compare)
├─ Oráculo (/oracle) – painel dentro da Arena + tela própria
└─ Forja (/forge)
```

O **Centro de Segurança** é um overlay sempre acessível (ícone fixo com a cor da VPN), não só uma rota.

Roadmap de UI (na sua ordem de prioridade) — novos sprints

Depois do Sprint 5 (Arena) e 5.5/5.6 (Terminal/VPN), a segunda fase de UI:

```
Sprint 9 – Telas P1 (fecham preparação + contribuição científica + segurança visível).
  Central de Comando (/command) com preflight de VM/Docker/VPN/tools; Atlas do Ataque (/atlas) com
  estados de nó e filtros; Grimório (/grimoire) lendo cards/ + métricas; Centro de Segurança
  (/security) como overlay com kill-switch e auditoria. Use react-router; screens em ui/src/screens/.

Sprint 10 – Replay + Observatório.
  Linha do Tempo/Replay (/replay) reconstruindo o Field a cada evento do EventLog (aproveita o event
  sourcing – sem backend novo); Observatório (/metrics) com os dashboards do Sprint 6.

Sprint 11 – Evidências + Justiça + Oráculo (rigor profissional).
  Mesa de Evidências (/evidence) correlacionando artefatos tipados + cadeia de custódia; Sala da
  Justiça (/justice) como fila de validação sobre o agente Validador (5.5); Oráculo (/oracle) com o
  payload de explicabilidade da política LLM.

Sprint 12 – Laboratório de Alvos + Arena Comparativa.
  Laboratório (/labs) com a Docker control API (start/stop/snapshot/health); Arena Comparativa
  (/compare) sobre o runner de experimentos do Sprint 6. Sala do Cronista (/report) polindo o
  preview/export do relatório da Parte 2.5.
```

Prioridade que sigo (a sua): Central de Comando → Atlas → Grimório → Replay → Observatório → Evidências+Justiça → Laboratório → Arena Comparativa; com Centro de Segurança já no P1 e Oráculo quando a política LLM estiver pronta.

Dica de operação no VSCode

Trabalhe um sprint por vez em conversas/commits separados. Ao terminar um sprint, me peça: *"rode os testes deste sprint e me mostre o resultado; se algo falhar, corrija antes de seguir"*. Só gere os sprites no Codex quando o Sprint 5 estiver funcionando com placeholders — assim você vê o layout antes de investir na arte. Mantenha este guia aberto no VSCode: quando eu for ligar dois módulos, confiro a fronteira na Parte 0.1 antes de escrever a cola — é o que mantém tudo operacional e sem bug.